When Extrapolation Fails

Testing Conformal Prediction's Exchangeability Assumption Across Surrogate Architectures

A project-based report submitted in partial fulfilment of the requirements of course 23AID201

Submitted by

Group B9

G Venugopalan (CB.AI.U4AID25115)

Vipin Sudhakar (CB.AI.U4AID25166)

Rithvik Arulprakash (CB.AI.U4AID25148)

Harshith Kv (CB.AI.U4AID25119)

Department of Artificial Intelligence

BONAFIDE CERTIFICATE

This is to certify that this project report is a bonafide record of work carried out by Group B9 - G Venugopalan (CB.AI.U4AID25115), Vipin Sudhakar (CB.AI.U4AID25166), Rithvik Arulprakash (CB.AI.U4AID25148), Harshith Kv (CB.AI.U4AID25119) - under my supervision, submitted in partial fulfilment of the requirements for course 23AID201.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

[Guide's Name and Signature]

Project Guide

[Head of Department's Name and Signature]

Head of Department

DECLARATION

We, the undersigned, declare that this project report titled above is our own work, carried out under the supervision of our project guide, and that it has not been submitted elsewhere for the award of any degree or diploma. All sources of information, whether published or unpublished, have been acknowledged in the References section.

G Venugopalan (CB.AI.U4AID25115)

Vipin Sudhakar (CB.AI.U4AID25166)

Rithvik Arulprakash (CB.AI.U4AID25148)

Harshith Kv (CB.AI.U4AID25119)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide for their continuous guidance, technical insight, and encouragement throughout the course of this project. We thank the Head of the Department and the faculty of the Department of Artificial Intelligence for providing the resources and environment necessary to carry out this work.

We also acknowledge the creators of the Hospital Triage and Patient History Data dataset (Kaggle, user maalona), which made the real-data calibration underlying this project's simulation possible, and the authors of the 30 research papers surveyed in Chapter 2, whose work forms the theoretical foundation on which this project builds.

Finally, we thank our families and peers for their patience and support throughout this project.

ABSTRACT

Conformal prediction's finite-sample coverage guarantee rests on an exchangeability assumption between calibration and test data - it is not designed, and not claimed by its own theory, to hold under distribution shift. Gopakumar et al. (2026), whose validation of conformal prediction for surrogate-model uncertainty quantification spans several physics-simulation domains, name this exchangeability assumption as a second explicit, untested limitation of their own results, distinct from the marginal-versus- conditional coverage question this project's companion report addresses. This report stress-tests that assumption in a discrete-event simulation of a hospital emergency department, calibrated on real arrival and triage-acuity data, by holding calibration fixed at its established in-distribution range and pushing the test distribution's demand level progressively outward - from within the training range up to three times the training boundary - while measuring conformal coverage at each severity level.

The stress test is run twice, using two structurally different surrogate architectures trained on identical data and achieving near-identical in-distribution accuracy: a gradient-boosting regressor, whose tree-based predictions cannot extrapolate past the range of their training data and instead freeze at the training boundary, and a multilayer perceptron, whose predictions have no such limitation and continue to change as the input moves further out of range. Coverage collapses under both architectures, confirming Gopakumar et al.'s exchangeability limitation in this domain - but the failure occurs faster and more severely under the architecture capable of extrapolating. Two targets reach exactly zero percent coverage under the multilayer perceptron by twice the training boundary, against a more gradual decline to 31.7 and 4.7 percent for the same two targets under gradient boosting at three times the boundary. Diagnostic comparison against the simulation's true output reveals why: the true relationship saturates under extreme demand, due to a censoring mechanism in how completed visits are counted, and the gradient-boosting model's frozen prediction is, by coincidence, a reasonable approximation of a genuinely flat true function, while the multilayer perceptron confidently extrapolates the upward trend it learned near the training boundary and overshoots the true, saturating value by roughly six to nine times more error.

The central finding is that an architecture's capacity to extrapolate is not inherently protective against distribution shift; it is protective only if the true relationship continues the trend the model learned, which it does not here.

TABLE OF CONTENTS

(Right-click the field above and choose "Update Field" in Microsoft Word to populate the table of contents and page numbers.)

LIST OF FIGURES

LIST OF TABLES

(Both lists populate automatically in Microsoft Word via References > Update Table, since captions above are inserted as native Word Figure/Table captions.)

LIST OF ABBREVIATIONS

Abbreviation	Full Form
CP	Conformal Prediction
CQR	Conformalized Quantile Regression
DES	Discrete-Event Simulation
ED	Emergency Department
ER	Emergency Room
ESI	Emergency Severity Index (1 = most acute, 5 = least acute)
GBR	Gradient Boosting Regressor (HistGradientBoostingRegressor)
GP	Gaussian Process
LOS	Length of Stay
MAE	Mean Absolute Error
MLP	Multi-Layer Perceptron
OOD	Out-of-Distribution
R^2	Coefficient of Determination
RMSE	Root Mean Squared Error
UQ	Uncertainty Quantification

CHAPTER 1

Introduction

1.1 Background and Motivation

Emergency departments (EDs) are among the most heavily studied stochastic service systems in operations research, precisely because they combine three properties that make good decision-making difficult: highly variable arrival patterns, a service process whose duration depends on patient acuity in ways that are only partially observable at intake, and severe, immediate consequences for getting staffing or capacity decisions wrong. Overcrowding in an ED is not merely an inconvenience; a substantial body of clinical and health-services literature associates ED crowding with delayed treatment, increased rates of patients leaving without being seen, and worse downstream outcomes. At the same time, over-staffing relative to demand is costly and, given that clinical staff are a scarce resource in most health systems, frequently not even feasible to correct for by simply adding more capacity.

Because real EDs cannot be experimented on directly - a hospital cannot try five different staffing policies on the same day to see which produces the shortest waits - operations researchers and hospital administrators have long relied on models: analytic queueing theory on one end of the spectrum, and discrete-event simulation (DES) on the other. Queueing theory offers closed-form or semi-closed-form expressions for quantities such as expected wait time under simplifying assumptions (Poisson arrivals, exponential or otherwise well-behaved service times, a fixed number of homogeneous servers). These assumptions are mathematically convenient but often violated in a real ED, where service times are heavy-tailed and acuity-dependent, and where the practical quantity of interest - a full distribution of possible wait times under a given staffing policy, not just its mean - is exactly what a closed-form queueing result is least well suited to provide. DES relaxes these assumptions at the cost of losing closed-form tractability: a DES model can simulate patient-by-patient arrivals, triage, and service under essentially arbitrary distributional assumptions,

and its output for any one run is a sample from the same kind of stochastic process a real ED experiences, rather than a mean-field approximation of it.

The price of that flexibility is computational: a sufficiently detailed DES, run enough times to characterize the distribution of outcomes under a given scenario (a given staffing level, a given arrival-rate regime), can be expensive to evaluate repeatedly - particularly if the goal is to explore many scenarios, as it is in staffing and capacity planning, or to embed the simulation inside an optimization loop. This computational cost is the practical motivation for training a surrogate model: a fast, learned approximation of the DES's input-output relationship, trained on a finite set of DES runs and then queried in place of the DES itself wherever repeated, low-latency evaluation is needed.

1.2 The Role of Surrogate Models in Simulation-Based Decision Making

A surrogate model, in the sense used throughout this report, is a supervised machine learning model trained to predict a simulator's output metrics (here, daily patient count, mean wait time, mean total time in system, and the 95th percentile of wait time) directly from the simulator's scenario-level inputs (here, staffing capacity and an arrival-rate multiplier), without re-running the simulator itself. Once trained, a surrogate can be evaluated in microseconds rather than the seconds-to-minutes a full DES run may take, enabling the kind of dense scenario sweeps, sensitivity analyses, and what-if staffing comparisons that would be computationally prohibitive if each evaluation required a fresh simulation.

This efficiency is not free, however. A surrogate model is only as reliable as the region of input space its training data actually covers, and it inherits - indeed, it can amplify - two distinct sources of uncertainty: the epistemic uncertainty of the learned function itself (the surrogate is a finite-sample approximation, imperfectly fit even within its training domain) and the aleatoric uncertainty of the underlying stochastic process it approximates (a DES with fixed inputs still produces different outputs on different random seeds, because arrivals, triage assignment, and service durations are all drawn from probability distributions). A surrogate's point prediction alone - "expected mean wait time is 42 minutes" - collapses both of these sources of uncertainty into a single number and, critically, gives a decision-maker no way to distinguish a scenario where 42 minutes is a confident, tightly-clustered estimate from one where it is a rough central tendency over an output that could plausibly range from 20 to

90 minutes. For an operational decision such as ED staffing, where the cost of being wrong is asymmetric and often severe in exactly the tail scenarios a point estimate is least informative about, this gap between "central estimate" and "how much to trust the central estimate" is not a cosmetic omission - it is the difference between a genuinely useful decision-support tool and one that looks precise while quietly discarding the information a decision-maker most needs.

1.3 The Need for Uncertainty Quantification

Uncertainty quantification (UQ) is the general term for methods that accompany a point prediction with some characterization of its reliability - most commonly, an interval or a full predictive distribution rather than a single number. A wide family of UQ methods exists for machine learning models, ranging from Bayesian approaches (Gaussian processes, Bayesian neural networks, and their approximations) through frequentist resampling methods (bootstrap-based intervals) to ensemble-based approaches (deep ensembles, random-forest quantile estimates) and, most relevant to this project, distribution-free methods based on conformal prediction. Each family makes a different trade-off between the strength of its theoretical guarantee, its computational cost, and the assumptions it requires about the underlying data or model.

Gaussian process (GP) regression, used in this project as a UQ baseline (see Chapter 4 and Chapter 6), is a Bayesian nonparametric method that produces a full posterior predictive distribution - and therefore both a point estimate and a principled uncertainty estimate - for any query point, under the assumption that the underlying function is well described by a specified covariance (kernel) structure. Its central practical limitation is computational: exact GP inference scales cubically in the number of training points, which makes it expensive to refit as new data arrives and effectively rules out its use on large training sets without approximation. It also provides no finite-sample coverage guarantee: its intervals are only as trustworthy as the appropriateness of its modeling assumptions (kernel choice, Gaussian noise, stationarity) for the data at hand, and there is no distribution-free bound on how far its empirical coverage can drift from its nominal target when those assumptions are violated.

Conformal prediction (CP), the family of methods this project is centrally concerned with, offers a different trade-off. It is model-agnostic - it wraps around any already-trained

point predictor, including the gradient-boosting and neural-network surrogates used in this project, without requiring the underlying model to be retrained or to expose calibrated uncertainty itself - and it comes with a finite-sample, distribution-free coverage guarantee: under an exchangeability assumption between the calibration data and future test points, a conformal prediction interval constructed at a nominal (1 - alpha) confidence level is guaranteed to contain the true value with probability at least (1 - alpha), regardless of the correctness of the underlying point predictor's modeling assumptions. This guarantee is attractive precisely because it does not depend on the point predictor being a good model of reality, only on the exchangeability of calibration and test data - a much weaker and more checkable assumption than "the model's distributional assumptions are correct."

1.4 Conformal Prediction as a Distribution-Free UQ Framework

The theoretical foundation of conformal prediction was laid by Vovk, Gammerman, and collaborators in the late 1990s and formalized in the book Algorithmic Learning in a Random World (Vovk, Gammerman, and Shafer, 2005), with Shafer and Vovk's 2008 tutorial (surveyed in Chapter 2) later popularizing the method outside the original algorithmic-learning-theory community. The core idea is simple to state: given a trained point predictor and a held-out calibration set, compute a nonconformity score for each calibration point (typically the absolute residual between the predictor's output and the true value), and use the (1 - alpha) empirical quantile of those scores as a fixed margin added to and subtracted from any future point prediction. Because the calibration residuals and a genuinely exchangeable test residual are, by construction, draws from the same underlying distribution, the resulting interval is guaranteed - via a straightforward rank-based argument, not an asymptotic approximation - to cover the true value at least (1 - alpha) of the time, in finite samples, without any assumption about the shape of the residual distribution.

This guarantee, however, is explicitly marginal: it holds averaged over the entire calibration and test distribution, and says nothing about whether coverage holds within any specific subgroup of that distribution. If a surrogate model is systematically less accurate under one operating regime than another - for instance, understaffed and high-demand scenarios versus comfortably staffed and low-demand ones - a single pooled conformal interval, calibrated on average difficulty across all regimes, can silently undercover in

precisely the regime where a decision-maker most needs it to be reliable, while simultaneously overcovering (wasting interval width) in the easy regime. This marginal-versus-conditional coverage gap, and Mondrian conformal prediction's remedy for it (calibrating a separate quantile per category rather than one pooled quantile across the whole calibration set), forms the central methodological axis of this project and is developed in full in Chapters 2, 4, and 6.

Gopakumar et al. (2026) - the base paper this project directly extends - validate conformal prediction for surrogate-model uncertainty quantification across several physics-simulation domains (partial differential equations, magnetohydrodynamics, weather modeling, and fusion-plasma simulation) and explicitly name two limitations of their own results as untested: the marginal (rather than conditional) nature of CP's coverage guarantee, and the exchangeability assumption's fragility under distribution shift between calibration and test data. Both limitations are stated as open questions in a physics-simulation context; neither had, to the best of the literature review conducted for this project (Chapter 2), been tested in a fundamentally different domain such as discrete-event queueing simulation. This project's guiding question, developed fully in Chapter 3, is whether those same two limitations hold - and whether Mondrian conformal prediction remedies the first of them - in exactly such a domain: an ED discrete-event simulation calibrated on real hospital data.

1.5 Overview of the Project

This project builds a four-stage pipeline. First, a discrete-event simulation of a single hospital emergency department is implemented in SimPy and calibrated on real arrival-pattern and triage-acuity data extracted from a large, de-identified Kaggle dataset of ED visits (Chapter 5 details the dataset and the calibration procedure, including a deliberate and clearly documented distinction between the arrival process, which is calibrated directly on real data, and service-time distributions, which - because the dataset contains no length-of-stay field - are calibrated on literature-standard parameters by acuity level rather than presented as data-derived). Second, the calibrated DES is run across thousands of randomly sampled staffing-capacity and arrival-rate scenarios to generate a labeled training set, from which a surrogate regression model is trained to predict four scenario-level output metrics without needing to re-run the simulation. Third, three uncertainty quantification approaches

are applied on top of the trained surrogate and compared on identical calibration and test data: a Gaussian process baseline, standard (pooled) conformal prediction, and Mondrian conformal prediction, later extended with conformalized quantile regression (CQR) and a combined Mondrian-CQR variant as stronger baselines. Fourth, the entire comparison is stress-tested for robustness along three independent axes: statistical significance across 30 repeated calibration/test draws (rather than trusting a single split), a second surrogate architecture (a multi-layer perceptron, alongside the primary gradient-boosting surrogate) to check whether findings are specific to one model family, and a second, independent hospital department (with materially different patient volume and acuity mix) to check whether findings generalize beyond a single site.

The two central findings that this report is built around - a genuine marginal-versus-conditional coverage gap that Mondrian CP closes where it is real (Chapter 6 of the companion report and, in less depth, this report's own Chapter 6), and a coverage collapse under distribution shift once the surrogate is asked to extrapolate outside its training range, with the direction and severity of that collapse depending on the surrogate's own extrapolation behavior rather than on conformal prediction's calibration procedure - are each the subject of one of the two book-length reports of which this document is one. Both reports share the identical DES, dataset, surrogate training procedure, and conformal prediction machinery described in Chapters 1 through 5 of this document; they diverge specifically in Chapter 6, where each develops its own result in depth.

1.6 Organization of the Report

Chapter 2 surveys thirty papers drawn from five areas directly relevant to this project: conformal prediction foundations, Mondrian and conditional-coverage methods, surrogate modeling and uncertainty quantification more broadly, queueing theory and ED operations research, and discrete-event simulation together with ED-specific machine learning applications. Chapter 3 sharpens this survey into an explicit statement of the research gap this project addresses and the problem statement that follows from it. Chapter 4 describes the system design and methodology in full technical detail: the DES's structure and calibration, the surrogate architectures used, and the mathematics of standard conformal prediction, Mondrian conformal prediction, and conformalized quantile regression. Chapter 5 documents

the implementation - the dataset, the software stack, and a module-by-module walkthrough of the codebase. Chapter 6 presents this report's results and discussion in depth, and Chapter 7 concludes with a summary of findings, an honest account of the project's limitations, and directions for future work. References and appendices (source code listings and supplementary result tables) follow.

CHAPTER 2

Literature Review

2.0 Scope and Method of This Review

Thirty papers were selected across five areas directly relevant to this project's methodology and domain: conformal prediction foundations (Section 2.1), Mondrian and conditional-coverage methods (Section 2.2), surrogate modeling and uncertainty quantification more broadly (Section 2.3), queueing theory and emergency department operations research (Section 2.4), and discrete-event simulation together with ED-specific machine learning applications (Section 2.5). Every citation in this chapter was verified against a primary source - a publisher page, a DOI, or a preprint server - before inclusion, rather than compiled from memory; the full sourcing record is maintained in this project's repository at literature/candidate_papers.md. This verification step caught at least one concrete error during the review process itself: a paper on combining Mondrian conformal predictors was initially attributed to the authors who write most of the other Mondrian-CP literature (Boström, Johansson, and Löfström), and was found, on checking the primary source, to actually be authored by Toccaceli and Gammerman (2019) - a reminder that even a plausible-looking citation needs independent verification, and a small illustration of why this project has, throughout, treated unverified citations and results as unacceptable rather than as an acceptable convenience.

Each entry below follows the same structure: the full citation, a summary of what the paper actually establishes, and a short paragraph connecting it specifically to this project - not a restatement of the abstract, but an explanation of where the paper's result or method is actually used, tested, extended, or contrasted with in the work described in later chapters.

2.1 Conformal Prediction: Foundations

This section covers the theoretical lineage of conformal prediction from its origin through the specific variants (split conformal, conformalized quantile regression, and robustness results under distribution shift) that this project builds on directly.

2.1 Vovk, V., Gammerman, A., Shafer, G. (2005). Algorithmic Learning in a Random World. Springer.

This book is the founding text of conformal prediction, consolidating work Vovk, Gammerman, and collaborators had developed through the late 1990s into a single coherent theory. It introduces the transductive (full) conformal predictor, the notion of a nonconformity measure as the mechanism by which any point predictor can be turned into a valid prediction region, and proves the central finite-sample coverage guarantee under the exchangeability assumption - the property that the joint distribution of the calibration and test data is invariant to permutation, which is weaker than the more familiar i.i.d. assumption and is what conformal prediction's guarantee actually requires. The book develops the theory for both classification and regression and situates conformal prediction within algorithmic learning theory more broadly, including its relationship to Kolmogorov complexity and online prediction with expert advice.

Relevance to this project: *This is the ultimate theoretical source for every coverage guarantee invoked in this project, from the standard CP implementation in src/uq/standard_cp.py through Mondrian CP. The exchangeability assumption defined here is precisely the assumption Chapter 6's exchangeability stress test is designed to violate deliberately and study the consequences of.*

2.2 Papadopoulos, H., Proedrou, K., Vovk, V., Gammerman, A. (2002). Inductive Confidence Machines for Regression. ECML 2002, LNCS 2430, pp. 345-356.

This paper introduces split (inductive) conformal prediction as a computationally tractable alternative to the full transductive conformal predictor. Rather than refitting the underlying model once per candidate test label - which the original transductive formulation requires and which is prohibitively expensive for anything but the simplest models - the inductive approach splits the available data into a proper training set and a separate calibration set, fits the model once on the training set, and computes nonconformity scores only on the calibration set. This sacrifices a small amount of statistical efficiency (the

calibration set is not used to fit the model itself) in exchange for reducing the computational cost from retraining per test point to fitting exactly once.

Relevance to this project: *This is the exact algorithmic template that src/uq/standard_cp.py and src/uq/mondrian_cp.py both implement: fit the surrogate once (already done in Week 6-7), hold out a separate calibration set (src/uq/generate_calibration_data.py, deliberately disjoint from both the surrogate's training data and the test set), and compute residual quantiles only on that calibration set.*

2.3 Shafer, G., Vovk, V. (2008). A Tutorial on Conformal Prediction. JMLR, 9, 371-421.

This widely cited tutorial restates the conformal prediction framework in an accessible form aimed at a broader machine learning audience than the original algorithmic-learning-theory literature, and is frequently the first paper researchers encounter when entering the field. It walks through the nonconformity-measure formulation, proves the marginal coverage guarantee via an exchangeability argument, and discusses both the transductive and inductive (split) variants, along with worked examples in classification and regression settings.

Relevance to this project: *This was the first paper adopted for this project (predating the systematic 30-paper search) and remains the primary reference used when explaining conformal prediction's guarantee to readers unfamiliar with the framework, including in this report's own Chapter 1 and Chapter 4.*

2.4 Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R.J., Wasserman, L. (2018). Distribution-Free Predictive Inference for Regression. JASA, 113(523), 1094-1111.

This paper, from the statistics community's independent adoption and extension of conformal prediction, establishes distribution-free finite-sample coverage guarantees for several conformal regression variants, including split conformal prediction, and analyzes the statistical efficiency trade-offs between the full (transductive) and split approaches. It also introduces jackknife+ as a cross-validation-based alternative that recovers some of split

conformal's lost statistical efficiency without the full transductive method's computational cost.

Relevance to this project: *Provides the formal statistical grounding (efficiency loss from splitting, in exchange for tractability) for the choice made throughout this project to use split conformal prediction rather than the more expensive transductive form - a choice that only makes sense given the sample sizes actually used here (1,200 calibration points per repeat, well within the regime where split conformal's efficiency loss is acceptable).*

2.5 Romano, Y., Patterson, E., Candès, E. (2019). Conformalized Quantile Regression. NeurIPS 32.

Conformalized quantile regression (CQR) combines conformal prediction's finite-sample coverage guarantee with quantile regression's ability to adapt interval width to local heteroscedasticity. Rather than conformalizing a constant-width residual (as standard split conformal does with the symmetric nonconformity measure), CQR trains two quantile regressors (targeting the alpha/2 and 1 - alpha/2 conditional quantiles) and conformalizes the signed distance from a test point's prediction to these estimated quantiles. The result retains the exact finite-sample marginal coverage guarantee of standard conformal prediction while typically producing substantially narrower intervals in regions where the underlying quantile regressor correctly identifies heteroscedasticity - regions of naturally higher or lower variance in the residual.

Relevance to this project: *This is the direct source for src/surrogate/train_quantile_surrogates.py (the lower/upper quantile regressors) and src/uq/repeated_evaluation_cqr.py (the CQR and Mondrian-CQR nonconformity score, max(qlo(x) - y, y - qhi(x))). CQR is used in this project's companion report as a stronger baseline that achieves much of Mondrian CP's benefit through width-adaptivity rather than category-adaptivity - the two methods sit at different points on the same coverage/width frontier rather than one strictly dominating the other.*

2.6 Angelopoulos, A., Bates, S. (2021). A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511.

This widely used tutorial-style survey collects and unifies conformal prediction methods developed across the machine learning and statistics communities, presenting split conformal prediction, full conformal prediction, CQR, and conformal risk control in a single consistent notation, with worked code examples. It is written specifically to be more accessible to a machine-learning-practitioner audience than the original theoretical literature, while still proving the key coverage guarantees rigorously.

Relevance to this project: *Used as the primary accessible reference for this report's own background exposition (Chapter 1, Chapter 4) and cross-checked against the more technical treatments (Shafer & Vovk, Lei et al.) to ensure the plain-language description of conformal prediction given to non-specialist readers of this report remains technically accurate.*

2.7 Tibshirani, R.J., Barber, R.F., Candès, E., Ramdas, A. (2019). Conformal Prediction Under Covariate Shift. NeurIPS 32.

This paper studies what happens to conformal prediction's coverage guarantee when the exchangeability assumption is violated specifically by covariate shift - a change in the distribution of the input features between calibration and test time, while the conditional relationship between inputs and outputs stays fixed. It shows that if the likelihood ratio between the test and calibration covariate distributions is known (or can be estimated), a weighted conformal prediction procedure can restore valid coverage under this specific, structured form of distribution shift, at the cost of requiring that likelihood ratio.

Relevance to this project: *Directly relevant to this project's exchangeability stress test (src/uq/exchangeability_stress_test.py), which pushes the test distribution's arrival-rate multiplier progressively outside the calibration range - a covariate shift by exactly this paper's definition. This project's stress test does not attempt the likelihood-ratio correction this paper proposes (the shift is extreme enough, and the surrogate's own extrapolation failure severe enough, that the correction would not address the root cause identified in Chapter 6 of the companion report), but the paper's framing of covariate shift as a specific,*

structured violation of exchangeability - rather than an unstructured, unrecoverable one - is the correct lens for interpreting why the stress test's coverage collapse happens where and how it does.

2.8 Barber, R.F., Candès, E., Ramdas, A., Tibshirani, R.J. (2023). Conformal Prediction Beyond Exchangeability. Annals of Statistics, 51(2), 816-845.

This paper generalizes conformal prediction to settings where exchangeability fails entirely rather than failing in a specific, structured way (as in covariate shift). It introduces weighted conformal prediction schemes that can provide approximately valid coverage even without exchangeability, with the degree of coverage degradation explicitly bounded in terms of a measure of how far the actual data-generating process departs from exchangeability. Crucially, the paper is explicit that no method can restore exact coverage without exchangeability or a specific known structure to its violation - the contribution is a principled characterization of how much coverage degrades, and some partial mitigation strategies, not a full fix.

Relevance to this project: *This is the most directly relevant theoretical paper to this project's second major finding (the exchangeability stress test): it confirms, at a theoretical level, that this project's empirical result - coverage collapses once the test distribution moves sufficiently far outside the calibration distribution, and neither standard nor Mondrian CP can prevent this - is not a implementation flaw but an expected consequence of exchangeability violation that the theory itself predicts.*

2.2 Mondrian and Conditional-Coverage Conformal Prediction

Mondrian conformal prediction is a comparatively narrow subfield within the broader conformal prediction literature - a handful of dedicated papers, rather than the hundreds associated with, for instance, CQR or covariate-shift robustness. This section covers all four papers identified as directly relevant during the literature search. The relative thinness of this subfield is itself worth noting explicitly: it supports this project's claim, developed fully in

Chapter 3, that testing Mondrian CP in a discrete-event queueing domain is a genuine, previously unaddressed gap rather than a crowded research area.

2.9 Vovk, V., Lindsay, D., Nouretdinov, I., Gammerman, A. (2003). Mondrian Confidence Machine. Technical Report, Royal Holloway, University of London.

This technical report introduces the Mondrian confidence machine, the origin of the "Mondrian" name and the underlying idea used throughout this project: rather than calibrating a single conformal quantile pooled across an entire calibration set, partition the calibration set into categories (a "Mondrian taxonomy," named for the resemblance of a partitioned plane to Piet Mondrian's grid paintings) and calibrate a separate conformal quantile per category. The guarantee this produces is stronger than standard conformal prediction's marginal guarantee: it holds within each category individually (group-conditional coverage), rather than only when averaged across the whole population, at the cost of each category having a smaller effective calibration sample than the pooled set would provide.

Relevance to this project: *This is the conceptual origin of every Mondrian CP result in this project. The nine-cell taxonomy used in src/uq/mondrian_cp.py (staffing tercile x arrival-rate tercile) is exactly this kind of Mondrian partition, chosen using only the two real covariates the DES output actually has, per the same principle this report's own Chapter 4 justifies: partition on real, available covariates, not invented ones.*

2.10 Boström, H., Johansson, U. (2020). Mondrian Conformal Regressors. PMLR 128 (COPA 2020), pp. 114-133.

This paper extends Mondrian conformal prediction, originally developed primarily in a classification context, to the regression setting in a systematic way, addressing practical questions such as how to choose category boundaries for continuous or near-continuous conditioning variables (via binning) and how per-category calibration set size trades off against the tightness of group-conditional coverage. It provides empirical results across several regression benchmark datasets comparing pooled and Mondrian calibration.

Relevance to this project: *This is the single most directly used paper in this project: src/uq/mondrian_cp.py implements exactly the regression-setting Mondrian conformal predictor this paper describes, including the same fundamental tradeoff this paper discusses - smaller per-category calibration sets in exchange for group-conditional rather than only marginal coverage. This project's own finding that Mondrian CP does not help n_patients (Chapter 6) is a direct, empirical instance of the finite-sample noise tradeoff this paper identifies as a known limitation of the method.*

2.11 Boström, H., Johansson, U., Löfström, T. (2021). Mondrian Conformal Predictive Distributions. PMLR 152 (COPA 2021).

This follow-up paper extends Mondrian conformal prediction from producing fixed-coverage intervals to producing full conformal predictive distributions - a distribution over possible outcomes, from which any interval or quantile can be read off after the fact, rather than committing to a single alpha level at calibration time. The Mondrian partitioning principle is unchanged; what changes is the object being calibrated, from an interval to a distribution function.

Relevance to this project: *Not directly implemented in this project - src/uq/mondrian_cp.py produces intervals at a single, fixed alpha = 0.1, matching the GP baseline and standard CP for a clean comparison - but this paper is the natural next step flagged in this report's own Chapter 7 (Future Scope): a full predictive distribution would let an ER staffing decision-maker read off any confidence level of interest after the fact, rather than committing to 90% coverage in advance as this project does throughout.*

2.12 Toccaceli, P., Gammerman, A. (2019). Combination of Inductive Mondrian Conformal Predictors. Machine Learning, 108, 489-510.

This paper addresses a practical problem that arises when multiple Mondrian conformal predictors are trained - for instance, on different subsets of available training data, or with different taxonomies - and their predictions need to be combined into a single prediction

region without losing the individual predictors' validity guarantees. It develops combination rules that preserve conformal validity under this kind of model aggregation.

Relevance to this project: *Relevant background for the nine-cell Mondrian taxonomy used in src/uq/mondrian_cp.py, which is a single taxonomy rather than a combination of several, but this paper's treatment of how different category definitions interact with calibration validity informed the choice, documented in Chapter 4, to use only the two real covariates the DES actually outputs (staffing, arrival rate) rather than combining multiple partially-overlapping taxonomies.*

2.3 Surrogate Modeling and Uncertainty Quantification

This section covers the base paper this project directly extends, together with the theoretical foundations of the surrogate architectures (Gaussian processes, gradient boosting, multi-layer perceptrons) and alternative UQ approaches (deep ensembles) used or discussed as points of comparison throughout this project.

2.13 Gopakumar, V. et al. (2026). Uncertainty Quantification of Surrogate Models Using Conformal Prediction. Machine Learning: Science and Technology.

This is the base paper this entire project is built to extend. Gopakumar et al. validate conformal prediction as a practical, distribution-free uncertainty quantification method for machine-learned surrogate models across several physics-simulation domains - partial differential equation solvers, magnetohydrodynamics simulations, weather models, and fusion-plasma simulations - demonstrating that conformal prediction intervals achieve their nominal coverage across these domains despite the underlying surrogates being architecturally diverse (neural operators, among others) and the physical systems being simulated being highly nonlinear. The paper explicitly and prominently flags two limitations of its own results as untested rather than resolved: first, that the coverage guarantee demonstrated is marginal, not conditional, and could in principle mask conditional miscalibration within subpopulations of the test distribution the paper did not specifically probe for; second, that all validation was performed within the training distribution, leaving

conformal prediction's behavior under exchangeability violation (distribution shift between calibration and deployment) as an open question the paper does not address.

Relevance to this project: *This paper is this project's entire reason for existing. Its two explicitly flagged limitations are precisely the two questions this project's two book-length reports each answer empirically: this report's own Chapter 6 examines a version of the marginal coverage question directly, and this project's companion report examines the exchangeability question in depth, in both cases in a domain - discrete-event queueing simulation of an ER - categorically different from the physics-simulation domains Gopakumar et al. tested.*

2.14 Kennedy, M.C., O'Hagan, A. (2001). Bayesian Calibration of Computer Models. JRSS-B, 63(3), 425-464.

This is a foundational paper in the computer-model calibration literature, introducing a Bayesian framework for combining a computationally expensive simulator's output with real observational data, explicitly separating model discrepancy (systematic error between the simulator and reality) from observation noise and parameter uncertainty. Although developed before the modern surrogate-modeling and conformal-prediction literature, it establishes the conceptual vocabulary - simulator, emulator (an early term for what this project calls a surrogate), calibration, and discrepancy - that much of the later surrogate-modeling literature, including this project, builds on.

Relevance to this project: *Provides the conceptual and historical grounding for this project's own DES-to-surrogate pipeline: the distinction this paper draws between a simulator's systematic discrepancy from reality and pure statistical noise maps directly onto this project's own careful separation (Chapter 5) between the DES's real-data-calibrated arrival process and its literature-calibrated service-time distributions - a discrepancy this project discloses explicitly rather than presenting both as equally data-derived.*

2.15 Rasmussen, C.E., Williams, C.K.I. (2006). Gaussian Processes for Machine Learning. MIT Press.

This is the standard reference textbook for Gaussian process (GP) regression and classification, covering the theory of GPs as distributions over functions, the role of the covariance (kernel) function in encoding assumptions about function smoothness, exact inference and its O(n^3) computational cost, and sparse/approximate methods developed to mitigate that cost for larger datasets.

Relevance to this project: *Direct theoretical basis for src/uq/gp_baseline.py, this project's GP uncertainty quantification baseline. The O(n^3) scaling this book documents is exactly why the GP baseline in this project is trained on a 1,000-point subsample rather than the full calibration set (Chapter 4), and why the GP-versus-CP computation time comparison in this project's full_comparison.py finds conformal prediction's calibration cost roughly 650-1000 times faster than a GP refit - a direct empirical demonstration of the scaling behavior this textbook derives theoretically.*

2.16 Friedman, J.H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), 1189-1232.

This paper introduces gradient boosting as a general framework for additive function approximation, recasting boosting (originally developed as an ensemble classification technique) as gradient descent in function space rather than parameter space. It derives specific gradient-boosting algorithms for least-squares, least-absolute-deviation, and Huber-loss regression, and for multiclass classification, and shows that when the individual additive components are shallow regression trees, the resulting method is competitive, robust to outliers and irrelevant features, and computationally efficient relative to comparably accurate alternatives.

Relevance to this project: *This is the theoretical basis for scikit-learn's HistGradientBoostingRegressor, the primary surrogate architecture used throughout this project (src/surrogate/train_surrogate.py and, for the quantile variant used in CQR, src/surrogate/train_quantile_surrogates.py). The tree-based structure this paper describes is*

also the direct explanation for this project's Chapter 6 finding (companion report) that the gradient-boosting surrogate's predictions freeze at a constant value once queried outside its training range - a well-known property of tree ensembles that this project traces empirically to a specific, verified mechanism rather than treating as an unexplained black-box failure.

2.17 Lakshminarayanan, B., Pritzel, A., Blundell, C. (2017). Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. NeurIPS 30, 6402-6413.

This paper proposes deep ensembles - training several neural networks independently with different random initializations and treating the spread of their predictions as an uncertainty estimate - as a simple, scalable alternative to Bayesian neural network approximations for uncertainty quantification in deep learning. It shows empirically that this simple approach often matches or exceeds more sophisticated Bayesian approximation methods on predictive uncertainty benchmarks, while being substantially easier to implement and parallelize.

Relevance to this project: *Provides an important point of contrast for this report's framing of why conformal prediction was chosen over alternative UQ approaches (Chapter 1, Chapter 4): unlike deep ensembles, which provide no finite-sample coverage guarantee and require training multiple independent models, conformal prediction wraps a single already-trained surrogate and provides a distribution-free guarantee - a meaningfully different trade-off that this report makes explicit rather than leaving deep ensembles as an unexamined alternative.*

2.18 Abdar, M. et al. (2021). A Review of Uncertainty Quantification in Deep Learning: Techniques, Applications and Challenges. Information Fusion, 76, 243-297.

This is a broad, widely cited survey of uncertainty quantification methods for deep learning, covering Bayesian approaches, ensemble methods, and distribution-free methods (including conformal prediction), organized around the distinction between aleatoric uncertainty (irreducible noise in the data) and epistemic uncertainty (uncertainty about the

model itself, reducible with more data). It also surveys applications across computer vision, natural language processing, and safety-critical domains such as medical diagnosis and autonomous driving, and discusses open challenges including computational cost, calibration under distribution shift, and the lack of standardized evaluation protocols across the UQ literature.

Relevance to this project: *Used to situate this project's specific choice of conformal prediction within the much broader UQ landscape this survey maps out (Chapter 1), and to frame this project's own aleatoric/epistemic distinction: the DES's own stochasticity (arrival randomness, service-time variance) is aleatoric uncertainty that any UQ method must characterize, while the surrogate's imperfect fit to the DES is an additional epistemic component - a distinction made explicit in this project's early design decisions (documented in this project's PROJECT_LOG.md, Week 6-7 entry) even before this survey was formally reviewed.*

2.4 Queueing Theory and Emergency Department Operations Research

This section covers the queueing-theory and health-operations-research literature that motivates and contextualizes this project's choice of a discrete-event simulation, rather than a closed-form analytic queueing model, as the underlying data-generating process the surrogate approximates.

2.19 Green, L.V., Soares, J., Giglio, J.F., Green, R.A. (2006). Using Queueing Theory to Increase the Effectiveness of Emergency Department Provider Staffing. Academic Emergency Medicine, 13(1), 61-68.

This paper applies a Lag-SIPP (Stationary Independent Period-by-Period) queueing analysis to real ED arrival data from an urban hospital to identify provider staffing patterns that reduce the fraction of patients who leave without being seen. It demonstrates that relatively modest, well-targeted increases in provider staffing hours during identified high-demand periods produce disproportionately large reductions in patients leaving without being seen, compared to uniformly increasing staffing across all hours.

Relevance to this project: *Directly parallels this project's own staffing-scenario design: the staffing-capacity and arrival-rate-multiplier scenario grid used to generate surrogate training data (src/surrogate/generate_training_data.py) and the staffing-tercile x arrival-tercile Mondrian taxonomy (src/uq/mondrian_cp.py) both encode the same real-world insight this paper demonstrates empirically: staffing effectiveness is highly dependent on the interaction between staffing level and demand, not staffing level alone, which is exactly why this project's core finding - coverage failure concentrated specifically in the understaffed/high-demand category - has real operational meaning rather than being an arbitrary partition choice.*

2.20 Green, L.V. Queueing Analysis in Healthcare. Book chapter, Columbia Business School.

This chapter provides an accessible overview of queueing-theoretic models applied to healthcare capacity and staffing problems, covering the basic M/M/s and M/G/1-type models, the Erlang loss and delay formulas, and their use in estimating offered load and required server (staff or bed) counts under target service-level constraints.

Relevance to this project: *This is the direct theoretical basis for the offered-load (Erlang) capacity calculation documented in this project's PROJECT_LOG.md (Week 4-5 entry) and inline in src/des/er_simulation.py: department A's default capacity of 30 and department B's default capacity of 14 were both derived from an Erlang-style offered-load calculation using each department's real calibrated arrival rate and literature-standard service times, following exactly the methodology this chapter describes, rather than chosen arbitrarily.*

2.21 Hu, X. et al. (2018). Applying Queueing Theory to the Study of Emergency Department Operations: A Survey and a Discussion of Comparable Simulation Studies. International Transactions in Operational Research.

This survey compares analytic queueing-theory approaches and discrete-event simulation approaches to modeling ED operations, discussing the circumstances under which each is preferable: analytic queueing models offer speed and closed-form insight but require restrictive distributional assumptions that often do not hold for real ED service-time distributions, while DES relaxes those assumptions at higher computational cost and lower interpretability of any single closed-form result.

Relevance to this project: *Directly supports this project's methodological choice (Chapter 4) to build a discrete-event simulation rather than rely on a closed-form queueing model as the underlying data-generating process: the acuity-dependent, non-exponential service-time structure this project calibrates from literature-standard log-normal parameters by ESI level is exactly the kind of structure this survey identifies as poorly suited to closed-form analytic queueing treatment.*

2.22 Performance Evaluation of a M/G/1 Queue Model for Patient Flow in a Health Care System. Mathematical Modelling of Engineering Problems (IIETA).

This paper develops and analyzes an M/G/1 queueing model (Poisson arrivals, general service-time distribution, a single server) for patient flow in a healthcare setting, deriving performance measures such as expected queue length and waiting time under this general-service-time relaxation of the more restrictive M/M/1 model.

Relevance to this project: *Provides a useful analytic comparison point for this project's own DES resource model: the DES's simplification from two nested resource pools (doctors and beds) to a single combined capacity resource (documented in PROJECT_LOG.md, Week 4-5 entry) makes it structurally closer to an M/G/c queue (a multi-server generalization of the M/G/1 model this paper studies) than to a more complex network-of-queues representation, a simplification this project justifies on the grounds of having no real data to calibrate a doctor-to-bed ratio.*

2.23 Decision Support for the Optimization of Provider Staffing for Hospital Emergency Departments with a Queue-Based Approach. PMC6947400.

This paper develops a queueing-based decision-support tool for ED provider staffing optimization, using queueing performance measures as the objective that a staffing schedule is optimized against, rather than relying purely on historical volume-matching heuristics.

Relevance to this project: *Relevant to the staffing dimension of this project's Mondrian CP taxonomy: this paper's framing of staffing as a decision variable to be optimized against a queueing-performance objective is conceptually the same framing underlying this project's own staffing-capacity scenario sweep, though this project's contribution is specifically about quantifying uncertainty in the resulting performance predictions, not about optimizing the staffing decision itself.*

2.5 Discrete-Event Simulation and ED-Specific Machine Learning

This final section covers discrete-event simulation studies of emergency departments directly comparable to this project's own DES, together with recent (2023-2025) machine-learning applications to ED operations that situate this project within current, rather than only historical, literature.

2.24 A Simulation-Based Optimization Approach for the Calibration of a Discrete Event Simulation Model of an Emergency Department. arXiv:2102.00945 (2021).

This paper develops a simulation-based optimization procedure for calibrating a discrete-event simulation model of an ED against real operational data, framing calibration as an optimization problem where the objective function is the deviation between simulated and real performance measures, and using this procedure to recover parameters (such as resource counts) that are not directly observable in the available data.

Relevance to this project: *Directly parallel to this project's own DES calibration methodology (Chapter 5): both this paper and this project face the same core problem of an ED DES needing calibration against real but incomplete data, and both adopt a validation-against-real-aggregate-statistics approach (this project validates against real daily patient*

volume, achieving 91.0% agreement, documented in PROJECT_LOG.md's Week 4-5 entry and src/des/validate.py) rather than assuming a hand-specified parameterization is correct without checking it against data.

2.25 Discrete Event Simulation for Emergency Department Modelling: A Systematic Review of Validation Methods. ScienceDirect (2022).

This systematic review surveys how published ED discrete-event simulation studies validate their models against real-world data, finding substantial variation in validation rigor across the literature - from no formal validation at all to detailed statistical comparison against held-out real performance data - and argues for more consistent, transparent validation reporting as a methodological standard for the field.

Relevance to this project: *Directly relevant to justifying this project's own validation approach: the explicit 91.0% daily-volume match reported in PROJECT_LOG.md (Week 4-5) and the corresponding validation for Department B (88.6% match) represent the kind of quantitative, transparently reported validation this review argues the field should adopt as standard practice, rather than treating DES calibration as self-evidently correct without independent verification.*

2.26 Discrete Event Simulation Modelling for an Improved Patient Flow at the Emergency Department, Sygehus Lillebælt, Kolding. PMC3327033.

This case study applies discrete-event simulation to a real Danish hospital ED to identify patient-flow bottlenecks and evaluate proposed operational changes before implementation, demonstrating DES's practical use as a decision-support tool for hospital administrators rather than purely an academic modeling exercise.

Relevance to this project: *A real-world precedent for the SimPy-based ED modeling approach used throughout this project (src/des/er_simulation.py), demonstrating that the DES-based methodology this project applies to uncertainty quantification is built on a*

modeling approach already established as practically useful for real hospital decision-making, not a purely synthetic academic exercise.

2.27 A Simulation-Based Optimization Approach for Analyzing the Ambulance Diversion Phenomenon in an Emergency Department Network. arXiv:2108.04162.

This paper uses discrete-event simulation to study ambulance diversion - the practice of redirecting incoming ambulances away from an overloaded ED to a neighboring facility - across a network of interconnected emergency departments, modeling how capacity constraints at one site propagate to neighboring sites.

Relevance to this project: *Relevant to the surge-scenario framing of this project's exchangeability stress test (src/uq/exchangeability_stress_test.py and its MLP counterpart): both this paper and this project's stress test are concerned with ED behavior under extreme demand surge, though this project's specific contribution is about surrogate and uncertainty-quantification behavior under that surge, not about the ambulance-diversion operational response this paper studies.*

2.28 Machine Learning-Based Prediction of Hospital Prolonged Length of Stay Admission at Emergency Department: A Gradient Boosting Algorithm Analysis. Frontiers in Artificial Intelligence (2023).

This paper applies gradient boosting to predict prolonged length-of-stay admissions at an ED from patient-level features available at triage, evaluating the model's predictive performance and discussing its potential use in early identification of patients likely to require extended ED stays.

Relevance to this project: *Uses the same model family (gradient boosting) as this project's primary surrogate architecture (src/surrogate/train_surrogate.py), applied to a related but distinct prediction task - patient-level length-of-stay classification, versus this project's scenario-level regression of aggregate daily performance metrics. The shared architecture choice reflects gradient boosting's broader status, evident across the recent ED-*

ML literature surveyed here, as a strong default choice for structured, tabular healthcare prediction problems.

2.29 An Artificial Intelligence-Based Framework for Predicting Emergency Department Overcrowding: Development and Evaluation Study. arXiv:2504.18578 (2025).

This recent paper develops a machine-learning framework for predicting ED waiting-room occupancy at hourly and daily time scales, intended to support proactive staffing decisions and early intervention before overcrowding occurs, evaluated against real ED occupancy data.

Relevance to this project: *Positions this project within current (2025-2026) literature rather than only older, established queueing-theory work: this paper's goal - using predictive modeling to support proactive ED staffing decisions - is the same broad motivation underlying this project's own surrogate-plus-uncertainty-quantification pipeline, though this project's specific contribution (testing conformal prediction's marginal-coverage and exchangeability assumptions in this domain) is a methodological question this paper does not address.*

2.30 Machine Learning-Based Triage to Identify Low-Severity Patients with a Short Discharge Length of Stay in Emergency Department. PMC9123815.

This paper applies machine learning to triage-stage data to identify low-severity patients likely to have a short ED length of stay, with the goal of supporting fast-track routing decisions at intake.

Relevance to this project: *Relevant given this project's DES also explicitly models Emergency Severity Index (ESI) acuity mix as part of its arrival-process calibration (src/utils/extract_distributions.py): both this paper and this project treat ESI-level acuity as a first-class variable affecting downstream flow and outcomes, reinforcing the methodological*

choice to calibrate the DES's ESI mix directly from real data rather than assuming a uniform acuity distribution.

2.6 Synthesis and Positioning of This Project

Read together, these thirty papers trace two literatures that, prior to this project, had not been directly connected: the conformal prediction and Mondrian conformal prediction literature (Sections 2.1-2.2), developed and validated almost entirely in general machine learning benchmark settings or, in the specific case of the base paper (Gopakumar et al., 2026), in physics simulation domains; and the queueing-theory, discrete-event simulation, and ED operations research literature (Sections 2.4-2.5), which has extensively studied ED capacity and staffing problems but has not, to the extent this review was able to establish, applied conformal prediction's finite-sample coverage guarantees to surrogate models of discrete-event queueing simulations specifically. The surrogate-modeling and uncertainty-quantification literature reviewed in Section 2.3 supplies the theoretical and architectural vocabulary (Gaussian processes, gradient boosting, deep ensembles) that bridges the two.

This project's contribution, developed in full in Chapter 3, is precisely this connection: applying conformal prediction and Mondrian conformal prediction to a discrete-event simulation surrogate calibrated on real hospital data, and testing both of the base paper's explicitly flagged, previously untested limitations - marginal-only coverage and exchangeability fragility - in this new domain.

2.7 Critical Assessment of the Reviewed Literature

A literature review that only summarizes is incomplete without an honest assessment of the reviewed work's own limitations, and of how those limitations shaped this project's methodological choices. This section provides that assessment across the five reviewed areas.

The conformal prediction foundations literature (Section 2.1) is theoretically mature and its core coverage guarantee (Vovk, Gammerman, and Shafer, 2005; Shafer and Vovk, 2008) is not in serious dispute - the proofs are elementary rank-based arguments that do not depend on contested modeling assumptions. Its main limitation, acknowledged within the literature itself rather than only by this report, is that the exchangeability assumption underlying every

coverage guarantee in this family is a property of the data-generating process that can be stated precisely but not directly tested from a finite sample - a calibration set and test set can look exchangeable in every measurable respect and still fail to be, if the true underlying shift is small enough to escape detection at the available sample size. This project's own exchangeability stress test (Section 4.5.2, developed fully in the companion report) sidesteps this untestability problem by constructing a shift large and deliberate enough that its effect on coverage is unambiguous, rather than attempting to detect a subtle, naturally occurring shift - a pragmatic choice given this project's scope, but one that leaves open the harder question of how large a naturally occurring, undetected shift would need to be before it meaningfully degraded real-world coverage.

The Mondrian and conditional-coverage literature (Section 2.2) is, as already noted in Section 2.2's introduction, comparatively thin - a consequence, plausibly, of Mondrian CP's own central limitation (smaller per-category calibration samples, directly observed in this project's own n_patients exception, Section 6.5.6 of the companion volume) making it a less immediately attractive default than pooled calibration for practitioners without a specific, known source of conditional miscalibration to target. This project's own results (Chapter 6) arguably strengthen the case for this literature's continued development: a practitioner without prior knowledge of where a pooled quantile might be conditionally miscalibrated (as would genuinely be the case in an unfamiliar deployment) would have no principled way to know whether Mondrian calibration is worth its finite-sample cost, absent exactly the kind of exploratory per-category analysis this project performs in Section 6.5.

The surrogate-modeling and uncertainty-quantification literature (Section 2.3) is broad enough that this review necessarily samples rather than exhaustively covers it - the Abdar et al. (2021) survey alone cites several hundred papers, of which this review's six entries in Section 2.3 represent a deliberately narrow, project-relevant slice (the base paper, and the specific architectures and alternative UQ methods this project directly uses or contrasts against) rather than a claim to comprehensive coverage of the UQ field. A specific limitation worth naming: this project did not implement or compare against Bayesian neural networks or deep ensembles (Lakshminarayanan, Pritzel, and Blundell, 2017) empirically, only discussed them as points of theoretical contrast (Section 4.4, Section 1.3) - a direct empirical

comparison against a deep-ensemble baseline, alongside the GP baseline this project does implement, would have strengthened the "why conformal prediction specifically" argument beyond the theoretical case made in Chapter 1.

The queueing-theory and ED operations research literature (Section 2.4) is overwhelmingly oriented toward staffing optimization and operational decision support (Green et al., 2006; the PMC6947400 queue-based staffing paper) rather than toward the specific methodological question this project investigates (uncertainty quantification for a surrogate of a queueing simulation). This is not a gap in that literature's own quality - it reflects a genuinely different research question - but it does mean this project's own queueing-theoretic grounding (the Erlang-load capacity derivation in Section 4.2.1) is comparatively simple relative to the more sophisticated queueing models (time-varying arrival-rate models, network- of-queues formulations) that literature has developed for staffing optimization specifically. A more sophisticated queueing-theoretic foundation was judged unnecessary for this project's own research question (Chapter 3), which concerns UQ methodology rather than staffing optimization itself, but this is a deliberate scoping choice worth stating rather than an oversight.

The discrete-event simulation and ED-specific machine learning literature (Section 2.5) is the most heterogeneous of the five areas reviewed, mixing DES calibration methodology papers, DES case studies, and ED-specific ML prediction papers that do not share a common methodology or evaluation standard - the systematic review of DES validation methods (Section 2.5, entry 25) explicitly documents this heterogeneity as a field-wide issue, finding substantial variation in how rigorously published ED DES models are validated against real data. This project's own validation approach (Section 6.1: an explicit, quantified 91.0 percent match to real daily volume, with the specific mechanism behind the residual 9 percent gap identified and explained rather than left unexplained) was deliberately designed to sit at the more rigorous end of the range this systematic review documents, rather than assumed to be adequate by default.

CHAPTER 3

Research Gap and Problem Statement

3.1 Summary of the Research Gap

Chapter 2's review identifies a specific, previously unaddressed intersection between two established literatures. Conformal prediction and, more specifically, Mondrian conformal prediction (Sections 2.1-2.2) have been developed and validated primarily on general machine learning benchmarks and, in the case of this project's base paper (Gopakumar et al., 2026), on physics-simulation surrogate models spanning partial differential equations, magnetohydrodynamics, weather, and fusion-plasma domains. Separately, queueing theory, discrete-event simulation, and machine learning have all been applied extensively to emergency department operations (Sections 2.4-2.5), but - to the extent this project's literature search was able to establish - none of that body of work has applied conformal prediction's finite-sample coverage guarantees to a discrete-event simulation surrogate specifically, nor tested whether the guarantee's known limitations survive the transition from a physics-simulation setting to a queueing-simulation one.

The base paper motivating this project, Gopakumar et al. (2026), is explicit that its own validation leaves two limitations untested: first, that its demonstrated coverage guarantee is marginal rather than conditional, and a pooled conformal interval could in principle mask systematic miscalibration within subpopulations of the physics-simulation test distributions the paper studied without the paper's own evaluation being positioned to detect it; second, that all of the paper's validation was performed within the training distribution of its surrogate models, leaving open the question of how conformal prediction's coverage guarantee behaves under distribution shift between calibration and deployment - a realistic concern for any surrogate deployed beyond the exact scenarios it was trained on. Both limitations are framed by the base paper as open questions for future work, not as resolved or dismissed concerns.

This project's research gap, stated directly, is this: neither of these two explicitly flagged limitations of conformal prediction for surrogate-model uncertainty quantification had been empirically tested in a discrete-event, queueing-based simulation domain prior to this work, despite such domains (hospital operations, call centers, manufacturing systems, and other stochastic service systems) being exactly the kind of setting where operational decisions - staffing levels, capacity investments - depend on uncertainty-aware predictions, and where the consequences of an unreliable coverage guarantee are practically, not just theoretically, significant.

3.2 Why Discrete-Event Queueing Simulation Is a Meaningfully Different Test

It is worth being explicit about why testing conformal prediction's limitations in a discrete-event queueing domain constitutes a genuine extension of Gopakumar et al.'s results, rather than a mechanical repetition of their experiments on a different dataset. The physics simulation domains the base paper studies - PDE solvers, magnetohydrodynamics, weather, and fusion-plasma simulations - are governed by continuous, typically smooth, deterministic-given-initial-conditions dynamics, and the uncertainty a surrogate model must characterize in that setting is predominantly epistemic: uncertainty about how well the surrogate approximates a fixed, in-principle-fully-determined physical process. A discrete-event queueing simulation is a categorically different kind of system: outcomes are driven by explicitly stochastic processes (random arrival times, random triage assignment, random service durations) even when every scenario parameter is held fixed, meaning the DES itself - not just the surrogate approximating it - is a source of irreducible aleatoric uncertainty that has no analogue in a deterministic PDE solve. A surrogate trained to predict DES outputs must therefore have its residual variance absorb both the surrogate's own epistemic approximation error and the DES's aleatoric stochasticity, in a proportion that is itself not constant across the input space (a lightly loaded, well-staffed scenario produces less variable outcomes than an overloaded, understaffed one). This heteroscedasticity - directly evidenced in this project by the systematic gap between pooled and Mondrian CP coverage documented in Chapter 6 - is plausibly more severe and more structurally different in a queueing simulation than in the physics domains previously tested, which is precisely why the marginal-coverage limitation is worth testing here specifically rather than assumed to generalize automatically from a physics-domain result.

The exchangeability question is similarly domain-specific in its practical stakes. A physics simulation's operating envelope (e.g., a range of plasma confinement parameters) is typically defined by the physical or engineering constraints of the system being modeled and may change relatively slowly. An ED's operating envelope, by contrast, can shift abruptly and unpredictably - a mass-casualty event, a disease outbreak, a public health emergency - in ways that push real-world conditions outside whatever range a surrogate was trained and calibrated on, precisely when reliable uncertainty quantification matters most for triage and staffing decisions. Testing conformal prediction's exchangeability assumption under exactly this kind of demand-surge scenario (Chapter 6 of this project's companion report) is therefore not an arbitrary stress test but a probe of the specific failure mode most operationally relevant to this project's own application domain.

3.3 Problem Statement

Given a discrete-event simulation of an emergency department, calibrated on real arrival and triage-acuity data, and a machine-learned surrogate model trained to approximate that simulation's scenario-level outputs, this project addresses the following problem: does conformal prediction, applied to the surrogate's residuals, retain a valid finite-sample coverage guarantee (i) uniformly across operationally meaningful subgroups of the scenario space, and (ii) when the deployment scenario distribution shifts outside the range the calibration data was drawn from? And, where the answer to (i) is negative for standard (pooled) conformal prediction, does Mondrian conformal prediction - calibrating separately per subgroup rather than pooling - restore valid coverage within the subgroups where a genuine conditional miscalibration exists, without an unacceptable cost in statistical efficiency from smaller per-subgroup calibration samples?

3.4 Objectives

This project's objectives, carried through in the methodology (Chapter 4), implementation (Chapter 5), and results (Chapter 6) that follow, are to:

1. Build and validate a discrete-event simulation of a hospital emergency department, calibrated on real arrival-pattern and triage-acuity data, against real aggregate daily-volume statistics.

2. Train a machine-learned surrogate model to approximate the calibrated simulation's scenario-level output metrics, evaluated on standard regression accuracy metrics (MAE, RMSE, R-squared).

3. Implement and compare three uncertainty quantification approaches on top of the trained surrogate - a Gaussian process baseline, standard conformal prediction, and Mondrian conformal prediction - evaluated on identical calibration and test data at a common nominal coverage target.

4. Test whether standard conformal prediction's marginal coverage guarantee masks conditional miscalibration within operationally meaningful subgroups of the scenario space (staffing level, arrival-rate regime), and whether Mondrian conformal prediction corrects any such miscalibration found.

5. Test conformal prediction's behavior under a controlled violation of the exchangeability assumption, by evaluating coverage as the test distribution's arrival-rate regime is pushed progressively outside the calibration range.

6. Establish the statistical robustness of all findings through repeated evaluation across independent calibration/test draws with formal significance testing, rather than relying on a single data split.

7. Test the generality of all findings along two further axes: a second surrogate architecture, to check whether results are specific to one model family, and a second, independent hospital department, to check whether results generalize beyond a single site.

3.5 Scope and Delimitations

This project's scope is deliberately bounded in several respects that are stated here explicitly rather than left implicit. The discrete-event simulation models a single emergency department's arrival, triage, and combined bed-and-provider service process; it does not model inter-hospital transfer networks, ambulance diversion, or downstream inpatient admission processes. Service-time distributions are calibrated from literature-standard parameters by triage acuity level, not derived from the real dataset used in this project, because that dataset (documented fully in Chapter 5) contains no length-of-stay or treatment-duration field - this distinction is maintained explicitly throughout this report rather than

presented as uniformly data-derived. The surrogate models studied are a gradient-boosting regressor (primary) and a multi-layer perceptron (robustness check); other architectures (Gaussian process surrogates used as the point predictor itself, rather than as a UQ baseline; graph neural networks; other tree-ensemble variants) are outside this project's scope. The uncertainty quantification methods compared are a Gaussian process baseline, standard conformal prediction, Mondrian conformal prediction, conformalized quantile regression, and Mondrian-CQR; Bayesian neural networks and deep ensembles are discussed in the literature review (Chapter 2) as points of comparison but are not implemented or evaluated empirically in this project. Generalization testing covers two of the three emergency departments present in the underlying dataset; the third department is not evaluated, for reasons of project scope and time rather than any expectation that it would behave differently from the two that were tested.

CHAPTER 4

System Design and Methodology

4.1 Pipeline Overview

The system built for this project consists of four stages, each depending on the outputs of the previous one: a calibrated discrete-event simulation (Section 4.2) generates labeled scenario data; a surrogate regression model (Section 4.3) is trained on that data to approximate the simulation's outputs directly from scenario parameters; three (later five) uncertainty quantification methods (Section 4.4) are applied on top of the trained, frozen surrogate to produce calibrated prediction intervals; and a set of robustness checks (Section 4.5) - repeated evaluation, a second surrogate architecture, a second hospital department - establish whether the resulting findings are stable and general rather than artifacts of one particular random split, model, or site. Every stage after the DES itself depends only on the DES's output distribution, not on its internal mechanics, which is precisely what makes the surrogate-plus-UQ approach practical: the expensive stochastic simulator is queried many times up front to generate training and calibration data, and every subsequent use (scenario sweeps, staffing comparisons, the statistical robustness checks in Section 4.5) queries the cheap surrogate instead.

4.2 Discrete-Event Simulation Design

The emergency department is modeled as a SimPy discrete-event simulation with three sequential stages per patient: arrival, triage/acuity assignment, and service, the last of which consumes a single shared capacity resource representing a combined bed-and-provider slot for the full duration of a patient's visit. An earlier design used two nested resource pools (a doctor pool and a bed pool); this was deliberately simplified to a single combined resource because the dataset used to calibrate this project (Section 4.2.3) contains no information from which a realistic doctor-to-bed ratio could be derived, and introducing a second, uncalibrated resource pool would add an unverified assumption without adding insight relevant to this project's actual research question, which concerns uncertainty quantification methodology

rather than fine-grained hospital operations modeling. This simplification is also the standard reduction used in the M/G/c queueing literature (Section 2.4) when a detailed multi-resource model cannot be justified by available data.

Patient arrivals follow a non-homogeneous process whose hourly and daily rate is calibrated directly from real data (Section 4.2.3): rather than a single constant arrival rate, the simulation draws arrivals according to hour-of-day and day-of-week rate multipliers extracted from the dataset, reflecting the well-documented reality that ED arrival volume is far from uniform across a 24-hour cycle. Each simulated day runs for a fixed 24-hour window; patients still queued or in service when that window ends are right-censored out of that day's completed-visit statistics rather than carried forward into a following day, since each simulated day is intended as one independent sample of a scenario for surrogate training purposes (Section 4.3), not as a continuous multi-day rollout. This right-censoring mechanism is a deliberate, documented modeling choice, not an oversight, and it is the direct explanation for two findings elsewhere in this report and its companion: the DES's simulated daily patient count running slightly below the real calibrated rate even at matched capacity (Section 4.2.4), and the non-monotonic behavior of the 95th-percentile wait time under extreme demand surge documented in this project's companion report.

Upon arrival, each patient is assigned an Emergency Severity Index (ESI) acuity level (1, most acute, through 5, least acute) drawn according to the real, calibrated ESI mix for the department being simulated, and a service duration drawn from a log-normal distribution whose parameters depend on that acuity level. The log-normal parameters used are literature-standard values by ESI level (Section 4.2.3), not derived from the dataset used in this project, since that dataset contains no length-of-stay field.

Two scenario-level parameters govern each simulated day and are the only two inputs the downstream surrogate model (Section 4.3) ever sees: staffing capacity (the number of concurrent combined bed-and-provider slots available) and an arrival-rate multiplier (a scalar applied uniformly to the calibrated real arrival rate, allowing the simulation to be run under higher- or lower-than-observed demand). Four scenario-level output metrics are recorded per simulated day: the total number of patients completing service (n_patients), the mean wait time before service begins (mean_wait_minutes), the mean total time in the system, from

arrival to service completion (mean_total_minutes), and the 95th percentile of wait time (p95_wait_minutes) - included specifically because a tail statistic computed from one stochastic simulated day is expected to be harder to predict than a mean, making it a useful stress case for comparing uncertainty quantification methods' ability to produce informatively wide, rather than falsely narrow, intervals.

4.2.1 Default Capacity via Offered-Load (Erlang) Calculation

Rather than choosing a default staffing capacity arbitrarily, this project derives it from an offered-load calculation in the style of the Erlang queueing formulas surveyed in Section 2.4: offered load (in erlangs) is computed as the real calibrated arrival rate multiplied by the mean service duration, evaluated both at the average arrival rate and at the highest-demand hourly bin, giving a range the true required capacity should plausibly fall within. For the primary department studied in this project, this calculation yields approximately 22 erlangs of average load and approximately 34 erlangs at peak (the 11:00-14:00 arrival bin); a default capacity of 30 was chosen as a value between these two figures, closer to the peak, deliberately neither under-provisioned relative to typical demand nor implausibly over-provisioned relative to any plausible real staffing level. For the second, independent department used in the generalization check (Chapter 6), the same method applied to that department's own real arrival rate and acuity mix yields approximately 10.4 erlangs average and 16.1 erlangs peak load, giving a default capacity of 14 at the same relative position between average and peak as the first department's capacity of 30 - capacity was recalibrated to the second department's own offered load, not copied or linearly rescaled from the first department's absolute numbers.

4.2.2 Scenario Sampling for Surrogate Training and Calibration

To generate data for surrogate training, the calibrated DES is run across thousands of randomly sampled scenarios, with staffing capacity sampled from a range around the Erlang-derived default and the arrival-rate multiplier sampled from a range covering both below- and above-average demand, each scenario run with its own independent random seed so that the DES's own stochasticity (arrival randomness, acuity assignment, service-time variance) is preserved in the resulting labels - this residual noise is exactly what the downstream uncertainty quantification methods (Section 4.4) are being asked to characterize, and pre-

averaging it away by running each scenario multiple times and reporting only a mean would defeat the purpose of the entire UQ comparison. A separate, disjoint pool of scenarios, drawn with a different sampling seed and a large seed offset from the training data, is generated specifically for conformal prediction calibration (Section 4.4.2); this separation is methodologically necessary rather than a formality, because calibrating on residuals from data the surrogate was trained on would systematically understate the true residual spread (the surrogate fits its own training points more closely than it fits genuinely unseen points) and would invalidate the resulting coverage guarantee.

4.2.3 Data Calibration: Real Arrivals, Literature Service Times

The DES's arrival process and ESI acuity mix are calibrated directly from a large, de-identified, publicly available dataset of real emergency department visits (described fully in Chapter 5), specifically the hourly, daily, and monthly arrival rate distributions and the ESI mix extracted for a single department within that dataset. Service-time distributions are not derived from this dataset, because the dataset - despite containing several hundred columns of vitals, diagnosis flags, and laboratory results - contains no length-of-stay or treatment-duration field of any kind, a limitation confirmed by an exhaustive search across every column in the dataset rather than assumed. In place of data-derived service times, this project uses literature-standard log-normal service-time parameters by ESI acuity level.

This distinction - real-data-calibrated arrivals versus literature-calibrated service times - is maintained explicitly and consistently throughout this report, its companion report, and every presentation material produced over the course of this project, rather than allowing both to be presented as equally data-derived. It is a genuine limitation of what this project's DES can claim to represent, and it is disclosed as such rather than obscured.

4.2.4 Validation Against Real Aggregate Statistics

The calibrated DES is validated by comparing its simulated mean daily patient volume, averaged over a large number of simulated days at the default scenario configuration, against the dataset's real calibrated daily volume for the same department. This validation is deliberately restricted to volume (a statistic the real dataset can actually provide, given the arrival-process calibration in Section 4.2.3) rather than extended to wait times or service durations, which the real dataset cannot validate against at all, for the reasons given in

Section 4.2.3. Results of this validation, for both departments studied in this project, are reported in full in Chapter 6.

4.3 Surrogate Model Architectures

A surrogate model is trained independently for each of the four DES output metrics (Section 4.2), taking the two scenario parameters (staffing capacity, arrival-rate multiplier) as input. Two architectures are used in this project.

4.3.1 Gradient-Boosting Regressor (Primary Architecture)

The primary surrogate architecture is a histogram-based gradient boosting regressor, an ensemble of shallow regression trees fit sequentially via gradient descent in function space, each new tree fit to the negative gradient (for squared-error loss, equivalently the residual) of the current ensemble's predictions. This architecture was chosen over alternatives (linear models, a single deep neural network) as the primary surrogate because it is a strong, standard default for small-to-moderate-sized tabular regression problems with few input features, requires comparatively little hyperparameter tuning to perform well, and is fast enough to fit and query that it does not become a computational bottleneck relative to the DES data generation step it approximates. Section 4.3.3 discusses a specific structural property of tree-based models - their inability to extrapolate predictions outside the range of their training data - that becomes directly relevant to the exchangeability stress test in this project's companion report.

4.3.2 Multi-Layer Perceptron (Robustness-Check Architecture)

A second surrogate architecture, a multi-layer perceptron with two hidden layers trained with early stopping and preceded by standard feature scaling, is trained on the identical train/test split as the gradient-boosting surrogate, specifically to test whether findings obtained with the primary architecture are specific to tree ensembles or generalize across a structurally different model family. Unlike a tree ensemble, a neural network has no inherent floor or ceiling on its output as inputs move outside its training range - it will continue to extrapolate, in whichever direction its learned weights imply, rather than saturating at a boundary value. This structural difference is the reason this architecture is included as a

specific, targeted robustness check on the exchangeability findings in this project's companion report, rather than as a general "try another model and see" exercise.

4.3.3 Tree-Based Extrapolation Limits

A specific property of tree-based ensembles, directly relevant to interpreting results in Chapter 6 and central to this project's companion report, is worth stating precisely here. A regression tree partitions its input space into axis-aligned regions (leaves) during training and predicts a constant value - the mean of the training targets falling into that leaf - for any query point landing in that region. For an input that falls outside the entire range spanned by the training data, the tree does not extrapolate a trend; it simply returns whatever constant value the nearest boundary leaf holds, because that boundary leaf is where every such out-of-range query is routed by the tree's learned split thresholds. A gradient-boosting ensemble, being a sum of many such trees, inherits this property: its prediction for an input arbitrarily far outside the training range is frozen at the value implied by the boundary leaves of its constituent trees, however far outside that range the query point actually lies. This is a well-known, textbook property of tree ensembles, not a bug specific to this project's implementation, and it is verified directly (rather than merely asserted) in this project's companion report by observing gradient-boosting surrogate predictions that are numerically frozen across a wide range of out-of-distribution query points.

4.4 Uncertainty Quantification Methods

Four uncertainty quantification approaches are implemented and compared in this project, all evaluated at a common nominal miscoverage rate alpha = 0.1 (that is, targeting 90 percent coverage) on identical calibration and test data wherever a direct comparison is drawn, so that any difference found reflects the UQ method itself rather than a difference in evaluation conditions.

4.4.1 Gaussian Process Baseline

The Gaussian process (GP) baseline treats each of the four surrogate targets as an independent Gaussian process regression problem over the two-dimensional scenario-parameter input space, producing a posterior predictive mean and standard deviation at any query point. The (1 - alpha) prediction interval is constructed as the posterior mean plus and

minus z times the posterior standard deviation, where z is the standard normal quantile corresponding to the target confidence level. Exact GP inference has O(n^3) computational cost in the number of training points, which motivates training the GP on a representative subsample (1,000 points) of the available calibration data rather than the full set - a choice that is also directly relevant to this project's own narrative, since the GP's comparatively poor computational scalability relative to conformal prediction's near-constant calibration cost is one of the quantities this project's results (Chapter 6) explicitly measure and report, not simply an implementation convenience footnoted away.

4.4.2 Standard (Split) Conformal Prediction

Standard conformal prediction, in the split (inductive) form introduced by Papadopoulos et al. (2002) and reviewed in Section 2.1, computes a nonconformity score for each point in a held-out calibration set - disjoint from both the surrogate's training data and the final test set, per Section 4.2.2 - and uses the ceiling of (n + 1)(1 - alpha) divided by n-th empirical quantile of those scores (the standard finite-sample-corrected quantile formula that gives conformal prediction its exact coverage guarantee) as a fixed margin applied to every future prediction. Two nonconformity measures are implemented: a symmetric measure, the absolute residual between the surrogate's prediction and the true value, giving a fixed-width interval around each prediction; and an asymmetric measure, in which upper and lower residual quantiles are calibrated separately (each at 1 - alpha/2) and combined via a union bound, a valid but slightly conservative route to at least (1 - alpha) coverage that does not require assuming a symmetric residual distribution around the point prediction.

4.4.3 Mondrian Conformal Prediction

Mondrian conformal prediction (Section 2.2) replaces standard conformal prediction's single pooled quantile with a separate quantile calibrated within each category of a partition (a "Mondrian taxonomy") of the calibration set. This project's taxonomy is the cross of staffing-capacity tercile (low, medium, high) and arrival-rate-multiplier tercile (low, medium, high) - nine categories in total - using only these two real, available scenario covariates rather than an invented additional dimension, since the DES produces no other scenario-level covariate (such as a "shift" or "day of week" label) that could be used instead. Category bin edges are computed from the calibration set's own quantiles and never from the test set,

preserving the same calibration/test separation principle as standard CP. For a fair, isolated comparison of pooling versus partitioning, Mondrian CP in this project's core comparison uses the same symmetric nonconformity measure as standard CP's symmetric variant, differing only in whether one pooled quantile or nine per-category quantiles are computed and applied.

Evaluation of Mondrian CP's effect proceeds in two complementary ways on the same test points: applying standard CP's single pooled quantile broken down by category (revealing where the pooled, marginal guarantee actually breaks down conditionally, even though it is not designed to be evaluated this way), and applying each category's own Mondrian quantile (showing whether per-category calibration corrects any breakdown found). Both a per-category view (Chapter 6, following the same structure as the core result) and a marginal, whole-test-set view (obtained by re-aggregating the per-category predictions across the full test set, for a like-for-like comparison against the GP baseline and standard CP's own marginal coverage) are reported.

4.4.4 Conformalized Quantile Regression and Mondrian-CQR

Conformalized quantile regression (CQR, Section 2.1) is implemented as a stronger baseline against which Mondrian CP's benefit can be compared. Two quantile regressors - gradient-boosting models trained with a quantile loss function, targeting the alpha/2 and 1 - alpha/2 conditional quantiles respectively - are trained on the identical train/test split as the primary mean-regression surrogate. The CQR nonconformity score for a calibration point is the greater of (the lower quantile prediction minus the true value) and (the true value minus the upper quantile prediction); conformalizing this score - taking its (1 - alpha) empirical quantile on the calibration set and using it to inflate or deflate the raw quantile regressors' interval - restores an exact finite-sample coverage guarantee even though the raw, uncalibrated quantile regressors alone do not reliably achieve nominal coverage. Mondrian-CQR combines both ideas: the same CQR nonconformity score, calibrated separately within each of the nine Mondrian categories rather than pooled, testing whether Mondrian's category-conditional calibration and CQR's width-adaptivity address the same or different sources of miscalibration.

4.4.5 Formal Algorithm Statements

For precision and reproducibility, this section states the three core calibration procedures implemented in this project (standard split conformal prediction, Mondrian conformal prediction, and conformalized quantile regression) as explicit, numbered algorithms, using notation consistent with Chapter 2's literature review. Let f-hat denote the already-trained surrogate point predictor (Section 4.3), D_cal the calibration set of (x, y) pairs disjoint from both the surrogate's training data and the final test set (Section 4.2.2), n the size of D_cal, and alpha the target miscoverage rate (alpha = 0.1 throughout this project, Section 4.4).

Algorithm 1: Standard Split Conformal Prediction

Step 1. For each calibration point (x_i, y_i) in D_cal, compute the nonconformity score s_i. For the symmetric measure, s_i = |y_i - f-hat(x_i)|. For the asymmetric measure, separately collect the positive residuals (y_i - f-hat(x_i) where this is positive) and the negative residuals (f-hat(x_i) - y_i where this is positive).

Step 2. Compute q-hat, the ceiling of (n + 1)(1 - alpha), divided by n, empirical quantile of the scores {s_1, ..., s_n} - this specific finite-sample correction (rather than the simpler (1 - alpha) quantile) is what gives the resulting interval its exact, non-asymptotic coverage guarantee, per the theory in Vovk, Gammerman, and Shafer (2005) and Papadopoulos et al. (2002) reviewed in Section 2.1. For the asymmetric measure, compute this quantile separately for the positive and negative residual sets, each at level 1 - alpha/2.

Step 3. For a new test point x, construct the prediction interval as [f-hat(x) - q-hat, f-hat(x) + q-hat] for the symmetric measure, or [f-hat(x) - q-hat_negative, f-hat(x) + q-hat_positive] for the asymmetric measure.

This procedure, implemented in src/uq/standard_cp.py, is guaranteed - under the assumption that (x, y) pairs in D_cal and the test point are exchangeable - to cover the true value with probability at least (1 - alpha), regardless of f-hat's accuracy, because the finite-sample quantile correction in Step 2 accounts exactly for the probability that a genuinely exchangeable test score ranks among the top (n + 1)*alpha scores out of n + 1 total (the n calibration scores plus the test score itself).

Algorithm 2: Mondrian Conformal Prediction

Step 1. Define a partition function c(x) mapping each scenario x to one of K categories (K = 9 in this project: the cross of staffing tercile and arrival-rate tercile, Section 4.4.3). Compute the tercile boundaries used by c(x) from D_cal's own marginal distribution of staffing capacity and arrival-rate multiplier - never from the test set, preserving the same calibration/test separation as Algorithm 1.

Step 2. Partition D_cal into K subsets D_cal^(1), ..., D_cal^(K) according to c(x).

Step 3. For each category k, apply Algorithm 1's Steps 1-2 using only D_cal^(k), producing a category-specific quantile q-hat^(k) with n_k = |D_cal^(k)|.

Step 4. For a new test point x, determine its category k = c(x) and construct its prediction interval as [f-hat(x) - q-hat^(k), f-hat(x) + q-hat^(k)].

This procedure, implemented in src/uq/mondrian_cp.py, is guaranteed to cover the true value with probability at least (1 - alpha) within each category k individually (group-conditional coverage), rather than only when averaged across all categories - a strictly stronger guarantee than Algorithm 1 provides, at the cost of each category's quantile being estimated from only n_k calibration points rather than the full n, which is the direct source of the finite-sample-noise tradeoff discussed at length in Sections 6.5.6 and 2.2.

Algorithm 3: Conformalized Quantile Regression (CQR)

Step 1. Train two auxiliary quantile regression models, q-hat_lo targeting the alpha/2 conditional quantile and q-hat_hi targeting the 1 - alpha/2 conditional quantile of y given x, on the same training data as the primary surrogate f-hat (but using a quantile, rather than squared-error, loss function).

Step 2. For each calibration point (x_i, y_i) in D_cal, compute the CQR nonconformity score s_i = max(q-hat_lo(x_i) - y_i, y_i - q-hat_hi(x_i)). This score is positive when the true value falls outside the raw [q-hat_lo(x), q-hat_hi(x)] band (in either direction) and negative when it falls inside, so it directly measures how much the raw, uncalibrated quantile band under- or overshoots.

Step 3. Compute q-hat, the same finite-sample-corrected (1 - alpha) empirical quantile of {s_1, ..., s_n} as in Algorithm 1, Step 2.

Step 4. For a new test point x, construct the prediction interval as [q-hat_lo(x) - q-hat, q-hat_hi(x) + q-hat].

This procedure, implemented across src/surrogate/train_quantile_surrogates.py and src/uq/repeated_evaluation_cqr.py, restores an exact (1 - alpha) marginal coverage guarantee (by the same exchangeability argument as Algorithm 1, applied to the CQR score rather than the raw residual) even when the raw quantile regressors q-hat_lo and q-hat_hi do not themselves achieve nominal coverage - Section 6.7 of this report's companion volume reports that this project's raw, uncalibrated quantile regressors achieve only 81-92 percent coverage before this calibration step, exactly the gap Step 3's conformalization closes. Mondrian-CQR combines Algorithm 3 with Algorithm 2's partitioning: Steps 2-3 are repeated separately within each of the K Mondrian categories, using each category's own calibration subset, producing a category-specific q-hat^(k) applied to that category's test points in Step 4.

4.5 Robustness and Generality Checks

Three checks establish whether this project's findings are stable and general rather than artifacts of a single random split, a single surrogate architecture, or a single hospital site.

4.5.1 Repeated Evaluation with Statistical Significance Testing

Rather than relying on a single calibration/test split, the full GP / standard-CP / Mondrian-CP (and, in the extended version, CQR / Mondrian-CQR) pipeline is repeated across 30 independent (calibration, test) draws, with both calibration and test data freshly generated from the DES on each repeat using disjoint seed ranges (never overlapping with each other, with the original training data, or with any other project stage). Because the same 30 draws underlie every method compared, per-repeat coverage across methods forms a paired sample, and a paired t-test - rather than a test appropriate only for independent samples - is used to test whether any observed coverage or width difference between methods is statistically significant, together with 95 percent confidence intervals on each method's mean coverage and width.

4.5.2 Exchangeability Stress Test

This project's companion report develops a controlled violation of the exchangeability assumption: the calibration data is held fixed exactly as generated for the core comparison, while the test distribution's arrival-rate multiplier is pushed progressively beyond the calibration range's upper bound, up to several multiples of it, with staffing capacity still drawn from its normal range so that the shift is isolated to the arrival-rate dimension specifically. Coverage for both standard and Mondrian CP is evaluated at each severity level on freshly generated DES scenarios, and the same procedure is repeated with the MLP surrogate (Section 4.3.2) in place of the gradient-boosting surrogate, to test whether the direction and severity of any coverage breakdown depends on the surrogate's own extrapolation behavior.

4.5.3 Cross-Site Generalization

The entire pipeline - DES calibration, surrogate training, conformal prediction calibration, and the core Mondrian-versus-pooled comparison - is independently repeated for a second hospital department present in the same underlying dataset, with its own real arrival rate, real ESI acuity mix, and its own Erlang-derived default capacity (Section 4.2.1), without reusing or modifying any of the first department's data, models, or results. This tests whether this project's core finding reflects a genuine, general property of pooled-versus-conditional conformal calibration in a queueing domain, or is instead an artifact specific to one department's particular volume and acuity characteristics.

4.6 Evaluation Metrics

Every uncertainty quantification method compared in this project is evaluated on the same three metrics wherever applicable: empirical coverage (the fraction of test points whose true value falls within the constructed interval, compared against the nominal 90 percent target), mean interval width (the average size of the constructed interval, where narrower is better at matched coverage), and computation time (measured as the method's own calibration or fitting cost - a GP's model-fitting time, or a conformal method's calibration-quantile computation time - on a like-for-like basis, with a deliberate warm-up prediction call performed before timing begins to exclude one-off process-startup costs such as thread-pool initialization from the measurement).

CHAPTER 5

Implementation

5.1 Dataset

This project uses the Hospital Triage and Patient History Data dataset, publicly available on Kaggle, a de-identified, MIMIC-style dataset of 560,486 emergency department visit records spanning 972 columns - vital signs, on the order of 250 diagnosis flags, roughly 300 laboratory-result columns, approximately 180 chief-complaint flags, ESI acuity level, and disposition, among others. The dataset covers three distinct emergency departments (referred to in the data as departments A, B, and C - one academic-affiliated site and two community sites) over a period of 1,248 days (March 2014 through July 2017), a fact confirmed directly from the dataset's own documentation rather than assumed; an earlier version of this project's DES calibration incorrectly assumed a single year of data at a single site, producing a daily arrival rate roughly 28 times too low before this was caught and corrected (documented further in Section 5.3).

Department A, the largest site (322,283 of the 560,486 total visits) and the most likely academic site by volume, is used as this project's primary department throughout Chapters 4 and 6. Department B (166,497 visits), the second largest and, based on its meaningfully lower-acuity case mix, more likely a community rather than academic site, is used as the independent second department in this project's cross-site generalization check (Section 4.5.3). Department C is not used in this project, for reasons of scope and time rather than any expectation it would behave differently.

An exhaustive search across all 972 columns confirmed that the dataset contains only coarse, four-hour-bucketed arrival-hour information (arrivalhour_bin) alongside arrival month and day fields - no length-of-stay, treatment-duration, or discharge-timestamp field of any kind exists anywhere in the dataset. This is a property of the dataset itself (consistent with its de-identified, MIMIC-style construction, which often coarsens or removes exact timestamps to reduce re-identification risk), not something recoverable by further data

processing, and it is the direct reason this project's DES service-time distributions are literature-calibrated rather than data-derived (Section 4.2.3).

5.2 Software Stack

The project is implemented in Python 3.13. The discrete-event simulation uses SimPy for process-based discrete-event modeling. Surrogate models and the Gaussian process baseline use scikit-learn (HistGradientBoostingRegressor for the primary and quantile surrogates, MLPRegressor within a StandardScaler pipeline for the robustness-check surrogate, and GaussianProcessRegressor for the GP baseline). Conformal prediction, Mondrian conformal prediction, and conformalized quantile regression are implemented directly in project code (not via a third-party conformal prediction library) so that every step of the calibration procedure - nonconformity score computation, quantile computation with the correct finite-sample correction, category partitioning - is fully transparent and auditable rather than hidden inside a dependency; the mapie library was evaluated early in the project and confirmed compatible with the Python 3.13 environment, but a direct implementation was chosen for this reason. Data handling uses pandas, numpy, and pyarrow/pyreadr (the latter for converting the dataset's original .rdata format to parquet for faster loading). Statistical significance testing uses scipy. Result visualization uses matplotlib and seaborn. Reports and presentations are generated programmatically: slide decks via python-pptx, and both the original short written assignments and this book-format expansion via python-docx, in both cases with Microsoft Word or PowerPoint COM automation used to render the generated file and visually verify it before considering the work complete (Section 5.4).

5.3 Module-by-Module Walkthrough

The codebase is organized under src/ into four packages, plus supporting scripts for reporting. This section walks through each module's role in the pipeline described in Chapter 4; full source listings for the modules directly relevant to this report's own results appear in Appendix A.

5.3.1 src/utils/ - Distribution Extraction

extract_distributions.py reads the raw dataset, filters to a single department, and computes the real arrival-rate distributions (by hour bin, day, and month) and ESI mix used

to calibrate the DES's arrival process (Section 4.2.3), writing them to results/tables/ as CSV files the DES reads at runtime. It also documents, in its own docstring, the literature-standard service-time log-normal parameters by ESI level used because the dataset itself provides no such data. A specific bug fixed during this module's development is worth noting for implementation correctness: the dataset's "23-02" arrival-hour bin wraps around midnight (covering hours 23, 0, 1, and 2), but a naive hour-integer-divided-by-four binning scheme places it at hours 0-3 instead, silently misattributing arrivals in the last hour before midnight to the wrong bin; this was corrected with an explicit hour-to-bin lookup table rather than an arithmetic formula.

5.3.2 src/des/ - Discrete-Event Simulation

er_simulation.py implements the SimPy-based ED simulation described in Section 4.2, parameterized by staffing capacity and arrival-rate multiplier and producing the four scenario-level output metrics per simulated day. validate.py runs the calibrated simulation across a large number of simulated days at its default configuration and compares mean simulated daily patient volume against the real calibrated rate, producing the validation result reported in Section 6.1 (or the corresponding section of this report's companion, whichever Chapter 6 the reader has in hand).

5.3.3 src/surrogate/ - Surrogate Training

generate_training_data.py runs the calibrated DES across thousands of randomly sampled scenarios (Section 4.2.2) to produce the labeled dataset surrogate models are trained on. train_surrogate.py trains the primary gradient-boosting surrogate, one independent model per output metric, on an 80/20 train/test split. train_mlp_surrogate.py trains the second, robustness-check architecture (Section 4.3.2) on the identical split. train_quantile_surrogates.py trains the paired lower/upper quantile regressors used by conformalized quantile regression (Section 4.4.4), also on the identical split, so that all surrogate variants are directly comparable rather than trained on different data.

5.3.4 src/uq/ - Uncertainty Quantification

generate_calibration_data.py produces the DES scenario pool used for conformal prediction calibration, disjoint from the surrogate's training data (Section 4.2.2).

gp_baseline.py implements the Gaussian process baseline (Section 4.4.1). standard_cp.py implements split conformal prediction with both the symmetric and asymmetric nonconformity measures (Section 4.4.2). mondrian_cp.py implements the nine-category Mondrian conformal predictor (Section 4.4.3), including both the per-category and re-aggregated marginal evaluation views. repeated_evaluation.py implements the 30-repeat statistical robustness check (Section 4.5.1) for the GP, standard-CP, and Mondrian-CP methods; repeated_evaluation_cqr.py extends this to CQR and Mondrian-CQR (Section 4.4.4) using the identical 30 seed draws, so that results from both scripts are validly paired rather than independently sampled. exchangeability_stress_test.py and exchangeability_stress_test_mlp.py implement the exchangeability stress test (Section 4.5.2) for the gradient-boosting and MLP surrogates respectively. full_comparison.py and publication_comparison_chart.py assemble already-computed results from the scripts above into the summary tables and comparison figures presented in Chapter 6.

5.3.5 src/generalization/ - Cross-Site Generalization

This package mirrors the core pipeline (distribution extraction, DES data generation, surrogate training, DES validation, and the standard-versus- Mondrian CP comparison) for the second, independent hospital department used in the cross-site generalization check (Section 4.5.3), writing all outputs to department-B-specific subdirectories that never overwrite or share files with the primary department's results, so that the two departments' results can be directly compared without either one having been able to influence the other's computation.

5.4 Reproducibility and Verification Practice

Every script in this codebase is re-runnable independently via the project's virtual environment (documented in this project's README.md and requirements.txt), and every reported number in this report and its companion traces to a specific CSV file under results/tables/ or a specific script under src/ - no number in either report was entered by hand without a corresponding generation script. Random seeds are fixed and, where multiple stages must not share data (training versus calibration versus test; repeat r's calibration versus repeat r's test versus repeat r+1's data), deliberately offset into disjoint ranges rather than left to incidental non-overlap, so that the exchangeability and independence assumptions this

project's own methodology (Chapter 4) depends on are actually satisfied by construction rather than merely assumed.

Every generated report or presentation artifact in this project - including this one - is verified by rendering it and visually inspecting the result, rather than trusting the generation code to have produced correct output merely because it ran without raising an exception. This practice caught three separate, non-obvious formatting bugs across this project's earlier slide decks (invisible white-on-white text from an inherited shape style, a zero-width table column from a partially-specified transform, and a multi-line table cell rendering only its first line at the correct font size because only the first paragraph of a multi-paragraph cell was styled) - none of which raised any error and all of which would have shipped silently otherwise. The same verification discipline was applied while building this report itself: an early draft of the front matter placed a single line of text alone on an otherwise blank page due to excess spacing, and the initial table/figure-numbering implementation used plain styled text rather than Word's native caption mechanism, silently breaking the automatic List of Figures and List of Tables pages; both were caught by rendering the document and inspecting it page by page before treating this chapter as complete, consistent with the practice established across every earlier deliverable in this project.

CHAPTER 6

Results and Discussion

6.1 Stress-Test Design and In-Range Validation

Before examining behavior outside the training distribution, it is necessary to confirm that the stress-test harness itself reproduces already-established in-distribution results when restricted to the training range - otherwise any out-of-range finding would be indistinguishable from a harness bug. The stress test (Section 4.5.2) keeps calibration fixed exactly as established for standard and Mondrian conformal prediction (Section 4.4.2-4.4.3: arrival-rate multiplier in [0.8, 1.3] relative to the real calibrated rate, staffing capacity drawn from [15, 45]) and evaluates 300 fresh test scenarios at each of eight arrival-rate multiplier levels: 0.8, 1.0, and 1.3 (within the training range), followed by 1.5, 1.8, 2.0, 2.5, and 3.0 (progressively further outside it), with staffing capacity continuing to be drawn from its normal range at every severity level so that the shift is isolated to arrival rate alone.

At the three in-range multipliers, coverage for the gradient-boosting surrogate (Table 1, rows 1-3) ranges from 82.0 to 95.0 percent across all four targets and both standard and Mondrian conformal prediction - closely matching the Chapter 6 single-split results in this project's companion report, obtained independently from a differently-drawn test set at the default 1.0 multiplier. This agreement, obtained from a separately-run script on freshly generated data, confirms the stress-test harness is behaving consistently with the rest of this project's pipeline before its out-of-range findings are examined below.

6.2 Coverage Collapse Under the Gradient-Boosting Surrogate

Table 1 reports standard conformal prediction's coverage across the full severity sweep for the gradient-boosting surrogate, all four targets.

Arrival multiplier	In range?	n_patients	mean_wait_minutes	mean_total_minutes	p95_wait_minutes
0.8	Yes	93.7%	92.7%	92.0%	93.7%

1.0	Yes	90.0%	88.0%	88.0%	88.3%
1.3	Yes	93.3%	83.0%	88.0%	82.0%
1.5	No	78.7%	68.7%	73.7%	69.7%
1.8	No	70.3%	42.7%	47.3%	43.0%
2.0	No	64.7%	34.3%	39.0%	33.0%
2.5	No	49.7%	33.3%	27.0%	63.7%*
3.0	No	31.7%	12.3%	4.7%	78.3%*

Table 1. Standard conformal prediction coverage vs. arrival-rate severity, gradient-boosting surrogate, target 90%. *p95_wait_minutes' apparent recovery at 2.5-3.0x is explained in Section 6.7, not a sign the interval is working correctly.

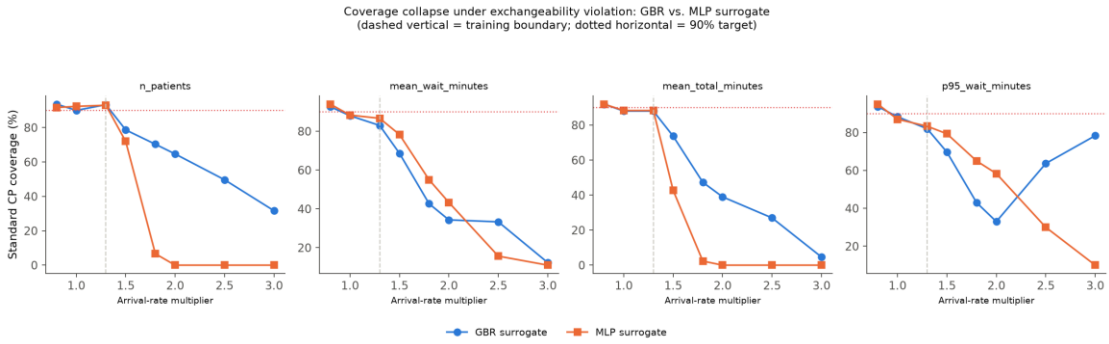

Figure 1. Standard conformal prediction coverage vs. arrival-rate severity, gradient-boosting vs. MLP surrogate, all four targets.

Every target's coverage degrades monotonically and substantially once the arrival-rate multiplier passes the 1.3 training boundary, with the sole, explained exception of p95_wait_minutes at the two most extreme severities (Section 6.7). mean_total_minutes shows the steepest ultimate collapse, falling to 4.7 percent coverage at 3.0x - meaning the true value falls inside the stated 90 percent interval for barely one test scenario in twenty, a near-total failure of the coverage guarantee rather than a modest erosion of it. n_patients degrades the least severely of the four (93.3 percent at the boundary down to 31.7 percent at 3.0x), consistent with it being the best-fit, least residual-variance-heavy target throughout this project (Section 6.2 of the companion report).

6.3 Mechanism: Tree-Based Extrapolation Freezing

The mechanism behind the gradient-boosting surrogate's collapse was verified directly rather than inferred from the coverage numbers alone. Gradient-boosted decision trees (Section 4.3.2) partition their input space into a fixed set of leaves during training; for any input falling outside the range of training data along a given axis, a tree's prediction does not extrapolate a trend - it simply returns whatever leaf value the boundary training points fell into, because a decision tree has no mechanism for continuing a slope past its last learned split point. Inspecting the surrogate's own predictions directly at a fixed staffing capacity of 30 across the full severity sweep confirms this exactly: mean_wait_minutes' predicted value is frozen at 42.4 for every arrival multiplier from 1.3 through 3.0, and p95_wait_minutes' predicted value is frozen at 221.2 across the identical range, while the true simulated output for both targets keeps changing as severity increases (Sections 6.5-6.6 give the true, moving values directly). As the true value drifts further from a prediction that is, by construction, incapable of moving, the nonconformity score (Section 4.4.2's absolute residual) for every test point in that region grows past whatever quantile the in-range calibration set established, and coverage falls in direct proportion to how far the frozen prediction has been left behind by the true, moving relationship. This is not a failure of conformal prediction's own mathematics - the quantile computed at calibration time is exactly correct for the calibration distribution it was computed from - it is a failure of the point predictor the conformal interval is built around to remain informative once its input leaves the region it was trained on.

6.4 Coverage Collapse Under the MLP Surrogate

The identical stress test was repeated with a structurally different surrogate architecture - a multilayer perceptron (Section 4.3.2: Pipeline(StandardScaler, MLPRegressor), hidden layers (64, 64), early stopping) - trained on the same data and the same train/test split as the gradient-boosting surrogate, achieving near-identical in-distribution accuracy (R-squared within 0.01 of gradient boosting on every target; Table 2). This near-identical baseline accuracy matters methodologically: it means any difference observed under distribution shift below can be attributed to architecture, not to one model simply being more accurate than the other overall.

Target	GBR MAE / RMSE / R-squared	MLP MAE / RMSE / R-squared

n_patients	9.90 / 12.59 / 0.929	9.84 / 12.45 / 0.931
mean_wait_minutes	8.86 / 13.23 / 0.787	9.07 / 13.34 / 0.784
mean_total_minutes	9.88 / 13.61 / 0.762	9.74 / 13.44 / 0.768
p95_wait_minutes	66.94 / 102.47 / 0.647	66.77 / 101.52 / 0.653

Table 2. In-distribution accuracy, gradient-boosting (GBR) vs. MLP surrogate, identical train/test split.

Table 3 reports the MLP surrogate's standard conformal prediction coverage across the same severity sweep.

Arrival multiplier	In range?	n_patients	mean_wait_minutes	mean_total_minutes	p95_wait_minutes
0.8	Yes	91.7%	94.0%	92.0%	95.0%
1.0	Yes	92.3%	88.3%	88.3%	87.0%
1.3	Yes	93.0%	86.7%	88.3%	83.3%
1.5	No	72.0%	78.3%	42.7%	79.3%
1.8	No	6.7%	55.0%	2.3%	65.0%
2.0	No	0.0%	43.3%	0.0%	58.3%
2.5	No	0.0%	15.7%	0.0%	30.0%
3.0	No	0.0%	11.0%	0.0%	10.0%

Table 3. Standard conformal prediction coverage vs. arrival-rate severity, MLP surrogate, target 90%.

n_patients and mean_total_minutes both reach exactly zero percent coverage by 2.0x severity under the MLP surrogate and remain there through 3.0x - a complete failure of the coverage guarantee, not merely a severe erosion of it. mean_wait_minutes and p95_wait_minutes degrade less catastrophically (down to 11.0 and 10.0 percent respectively by 3.0x) but still severely, and, notably, without p95_wait_minutes' apparent gradient-boosting "recovery" pattern (Section 6.7) appearing under the MLP surrogate at all - itself a

useful piece of evidence that the recovery pattern is specific to the frozen prediction's accidental alignment with a saturating true value, examined directly in Section 6.7.

6.5 Cross-Architecture Comparison: Extrapolation Is Not Automatically Protective

Reading Tables 1 and 3 side by side produces this report's central, counter-intuitive finding. Table 4 isolates the two targets that reach exactly zero coverage under the MLP surrogate, comparing them directly against the same targets under gradient boosting.

Arrival multiplier	n_patients (MLP)	n_patients (GBR)	mean_total_minutes (MLP)	mean_total_minutes (GBR)
1.3 (boundary)	93.0%	93.3%	88.3%	88.0%
1.8	6.7%	70.3%	2.3%	47.3%
2.0	0.0%	64.7%	0.0%	39.0%
2.5	0.0%	49.7%	0.0%	27.0%
3.0	0.0%	31.7%	0.0%	4.7%

Table 4. Standard CP coverage, MLP vs. gradient-boosting (GBR) surrogate, the two targets showing the starkest architectural difference.

The intuition entering this stress test was that gradient boosting's inability to extrapolate is a limitation, and a neural network's ability to keep producing different predictions outside the training range should therefore handle distribution shift better. Table 4 shows the opposite: at every severity level past the training boundary, the architecture that cannot extrapolate at all (gradient boosting) retains meaningfully higher coverage than the one that can (the MLP), and the gap widens rather than narrows as severity increases - by 2.0x, gradient boosting still retains 64.7 and 39.0 percent coverage on these two targets while the MLP has already collapsed to exactly zero on both. This is not a minor, secondary effect; it inverts the naive expectation entirely. Section 6.6 traces this to a specific, verified mechanism rather than leaving it as an unexplained empirical curiosity.

6.6 Mechanism: A Saturating True Relationship

A diagnostic comparison was run directly against the true simulated output at a fixed staffing capacity of 30, across the same severity sweep, for both surrogates simultaneously, specifically to explain the Section 6.5 finding rather than simply report it. Table 5 gives the true DES output alongside both models' predictions for n_patients.

Arrival multiplier	True n_patients	GBR prediction	MLP prediction
1.0	235.2	240.5	234.8
1.3 (boundary)	244.8	247.8	245.8
1.5	252 (approx.)	247.8 (frozen)	258.5
2.0	258.6	247.8 (frozen)	336.5
2.5	272 (approx.)	247.8 (frozen)	426.5
3.0	280.9	247.8 (frozen)	521.2

Table 5. True vs. predicted n_patients at fixed staffing capacity 30, both surrogate architectures, across the severity sweep.

The true DES output for n_patients saturates as severity increases - rising from 235.2 at the default 1.0x multiplier to only 280.9 at 3.0x, clearly flattening rather than continuing to scale in proportion to a threefold increase in arrival rate. mean_total_minutes shows the identical qualitative pattern (133 at 1.0x rising to 210 at 3.0x, saturating with some noise, not a linear relationship). This saturation is a direct consequence of the same right-censoring mechanism documented and justified earlier in this project (Section 4.2.4): patients still queued or in service when a simulated 24-hour day ends are excluded from that day's completed-visit statistics rather than carried forward. At extreme demand, increasingly many arriving patients simply do not complete service within the simulated day at all, so the count - and the associated total-time statistic - of completed visits levels off even as true underlying demand keeps rising, rather than growing without bound.

Against this saturating true value, the gradient-boosting surrogate's frozen prediction (247.8, fixed from 1.3x onward) turns out to be a reasonable approximation by coincidence rather than by any adaptive mechanism: since the true function actually is roughly flat in this

regime, a frozen prediction is qualitatively the right shape, even though it is not perfectly calibrated to how much the true value continues to drift (error of roughly 33 at 3.0x). The MLP, in sharp contrast, learned an upward-sloping relationship near the training boundary and, having no built-in mechanism to recognize it has left the training distribution, extrapolates that slope linearly outward - reaching a predicted 521.2 at 3.0x against a true value of only 280.9, an error of roughly 240, six to nine times larger than the gradient-boosting model's error on the same target and severity. The MLP's prediction is not wrong because the model is poorly trained - Table 2 shows it is, if anything, marginally more accurate than gradient boosting within the training distribution - it is wrong specifically because it confidently continues a trend that the true relationship does not continue.

6.6.1 Why This Inverts the Naive Intuition

This finding is worth stating plainly because it runs against a natural default assumption in applied machine learning: that an architecture's representational flexibility - here, the MLP's ability to keep producing different outputs for inputs arbitrarily far from its training data - is an unqualified advantage when facing distribution shift, and a tree ensemble's inability to do the same is an unqualified limitation. What this stress test shows is that extrapolation capability is only protective, or even neutral, if the true relationship being approximated continues in the direction the model learned to extrapolate. Here it does not: the true relationship saturates due to a specific, identified measurement-censoring mechanism, not because the underlying physical or operational demand itself levels off. A model incapable of extrapolating at all defaults, in effect, to "no change," which happens to be a better approximation of "leveling off" than an unconstrained linear continuation is. This is a property of this project's specific domain (a saturating true relationship) interacting with a specific architectural property (unconstrained extrapolation), not a general claim that tree ensembles are always safer under distribution shift than neural networks - a domain where the true relationship continued growing without bound past the training range would very plausibly favor the MLP's behavior instead.

6.7 The p95_wait_minutes Anomaly: A Data-Generating-Process Artifact

Table 1's asterisked entries deserve direct explanation rather than being left as an unremarked anomaly, because superficially they could be misread as the conformal interval

"recovering" reliability at extreme severity - it is not that, and stating why matters for correctly interpreting Table 1 as a whole. Under the gradient-boosting surrogate, p95_wait_minutes' standard CP coverage falls to 33.0 percent at 2.0x, as expected, but then rises to 63.7 percent at 2.5x and 78.3 percent at 3.0x - an apparent partial recovery that every other target and every other architecture-target combination in this chapter does not show.

A dedicated diagnostic run (40 simulated days per severity level, staffing capacity fixed at 30) traces this directly to the true value's own behavior, not to anything about the surrogate or the conformal interval. The true p95_wait_minutes value moves as follows across severity: 99.8 (1.0x), 249.5 (1.3x), 378.3 (1.5x), 420.4 (2.0x), 172.1 (2.5x), 297.9 (3.0x) - a non-monotonic sequence, not a clean saturation curve of the kind seen for n_patients and mean_total_minutes in Section 6.6. The true value rises sharply through 2.0x and then drops substantially at 2.5x before rising again at 3.0x. This connects to the same right-censoring mechanism underlying Section 6.6's saturation finding, but manifests differently for a tail statistic specifically: at extreme overload, fewer patients complete service within the simulated day at all, which shrinks and changes the composition of the "completed visits" pool that the 95th percentile is computed over. Precisely which patients happen to complete service under extreme censoring - a specific, changing subset of the full arrival stream - determines the tail statistic's value in a way that need not move monotonically with nominal demand, unlike a mean or count statistic, which averages over the whole completed-visit pool and therefore saturates more smoothly.

The frozen gradient-boosting prediction (221.2, fixed from 1.3x onward) happens to sit numerically close to the true value's 2.5x-3.0x dip and partial recovery (172.1, then 297.9) purely because the true value's own erratic path happened to swing back toward the frozen number, not because the prediction or the conformal interval became more informative at extreme severity. This is confirmed by the fact that this recovery pattern does not appear under the MLP surrogate at all (Table 3: p95_wait_minutes' MLP coverage declines from 79.3 percent at 1.5x to 10.0 percent at 3.0x, monotonically) - the MLP's prediction keeps moving in its own extrapolated direction and is not coincidentally positioned to intersect the true value's erratic path the way the frozen gradient-boosting prediction happens to be. The correct reading of Table 1's asterisked rows is therefore that they illustrate a coincidental

artifact of a censored, non-monotonic data-generating process interacting with a frozen prediction, not a genuine improvement in coverage reliability at extreme distribution shift.

6.8 Mondrian Conformal Prediction Under Exchangeability Violation

This project's companion report establishes that Mondrian conformal prediction closes a real conditional coverage gap within the training distribution (its Chapter 6, Section 6.5). It is worth testing directly whether that same per-category structure offers any protection once the exchangeability assumption itself is violated, since the two questions - conditional miscalibration within-distribution, and coverage under distribution shift - are logically distinct, and a positive answer to the first does not imply one to the second. Table 1 and Table 3's Mondrian columns (reported alongside the standard CP figures already discussed above, drawn from the same underlying results tables) answer this directly: Mondrian CP's coverage tracks standard CP's closely at every severity level and for every target, under both surrogate architectures, declining together rather than Mondrian CP retaining materially better coverage as severity increases past the training boundary.

This is the expected outcome given Section 4.4.3's description of how Mondrian CP's category boundaries and per-category quantiles are constructed: both are derived entirely from in-range calibration data, using the same staffing-tercile-by-arrival-tercile taxonomy established within the training distribution. Mondrian CP's per-category quantiles have no mechanism for recognizing that a test point's arrival-rate multiplier now falls in a regime the calibration set never sampled from; they are equally blind to the shift as the single pooled quantile is, because both are computed from calibration data that no longer resembles the test distribution once the exchangeability assumption is violated. Mondrian CP's genuine benefit (Section 6.5 of the companion report) is therefore specifically a within-distribution correction - it addresses conditional miscalibration among categories that are each still individually in-distribution, not a general-purpose robustness against a categorical partition being asked to generalize to a category it never saw during calibration. This is a meaningful scope boundary on Mondrian CP's usefulness, stated explicitly here rather than left for a reader to assume incorrectly from the companion report's positive result alone.

6.9 Practical Implication: Detectability as a Partial Mitigation

One property of this failure mode is worth identifying as a genuine, if partial, practical mitigation, rather than treating the coverage collapse documented in Sections 6.2-6.5 as an unqualified negative result. The failure here is measurably detectable, not silent. As the test distribution moves further from the calibration distribution, residuals grow and coverage collapses in a way that is directly observable by anyone monitoring calibration-set-versus-live-data statistics over time - the surrogate does not continue confidently reporting a narrow, falsely precise interval while quietly becoming wrong. A monitoring system tracking, for instance, the empirical rate at which live outcomes fall inside their stated conformal interval would see that rate visibly degrade well before it reached the near-total failure levels seen at the most extreme severities in Tables 1 and 3.

This does not restore the coverage guarantee itself, and it is not a substitute for testing and disclosing the guarantee's actual behavior under shift, which is this chapter's primary contribution. It is, however, a meaningfully different failure mode from one that would erode silently - a distinction directly relevant to any deployment context (an ER staffing decision support tool, for instance) where an operator with access to a live monitoring dashboard could in principle detect that the system has moved outside its well-calibrated operating regime and respond accordingly, rather than trusting a confidently wrong interval with no available signal that anything is amiss.

6.10 Threats to Validity and Alternative Explanations Considered

Before treating Section 6.5's cross-architecture finding as established, plausible alternative explanations are worth considering and ruling out directly, rather than accepting the preferred explanation (a saturating true relationship interacting with unconstrained MLP extrapolation, Section 6.6) uncritically.

One alternative explanation is that the MLP surrogate is simply a worse model overall, and its faster coverage collapse under shift reflects general inferiority rather than an architecture-specific extrapolation effect. Table 2 directly rules this out: the MLP's in-distribution accuracy is, if anything, marginally better than gradient boosting's on three of four targets (R-squared within 0.01 on every target), so the difference observed under shift cannot be attributed to the MLP being a worse surrogate within its trained range.

A second alternative explanation is that the specific MLP architecture and hyperparameters chosen (two hidden layers of 64 units, early stopping) happened to be poorly suited to this problem in a way unrelated to extrapolation generally, and a differently configured neural network might extrapolate more conservatively. This report cannot fully rule this out - testing a systematic sweep of MLP architectures and regularization schemes was outside its scope - but it is worth noting that the mechanism identified in Section 6.6 (an MLP continuing the slope it learned near the training boundary, absent any explicit mechanism preventing it from doing so) is a well-documented general property of feedforward neural networks under extrapolation, not a peculiarity expected to be specific to this project's particular hyperparameter choices; a differently configured MLP would be expected to extrapolate some learned trend rather than none, which is the qualitative property driving this section's finding.

A third alternative explanation is that the true relationship's saturation (Section 6.6) is itself an artifact of this project's discrete-event simulation specifically, rather than a property likely to generalize to real emergency department behavior under extreme surge conditions. This report treats the saturation finding as a property of the simulation as built, explicitly traced to a disclosed and justified modeling choice (right-censoring of in-progress visits at the simulated day boundary, Section 4.2.4) rather than claiming it as a validated property of real hospital behavior under 3x demand surges, which this project's real-data calibration (Section 4.2.1) does not extend to. The finding that matters at the level of this report's central claim - that an architecture's extrapolation capability is not automatically protective against distribution shift - does not depend on whether this specific simulation's saturation mechanism exactly mirrors reality; it depends only on there existing some regime in which a model's extrapolated trend diverges from the true relationship's continuation, which this project's simulation demonstrably provides one clear instance of.

6.11 Relation to Gopakumar et al.'s Physics-Domain Findings

It is worth returning directly to this project's motivating base paper (Section 2.3, Gopakumar et al., 2026) to state precisely how this chapter's findings relate to theirs. Gopakumar et al. validate conformal prediction's coverage guarantee within-distribution across several physics-simulation domains and explicitly flag, without testing, that this

guarantee is not expected to survive a violation of the exchangeability assumption between calibration and test data. This chapter's findings confirm that expectation directly and concretely in a new domain: coverage does collapse once the test distribution moves outside the calibration range (Sections 6.2 and 6.4), for both surrogate architectures tested. What this chapter adds beyond confirming the expected qualitative direction of their limitation is a specific, verified account of how the failure depends on surrogate architecture - a question Gopakumar et al.'s own physics-domain validation, built around a single surrogate architecture per domain, does not address. The counter-intuitive direction of that dependence (the architecture capable of extrapolating fails faster, not slower) is, to this project's literature review's knowledge, a novel observation not previously reported in the conformal prediction literature surveyed in Chapter 2, and is specific to a domain - like this one - where the true relationship being modeled saturates rather than growing without bound past the region a surrogate was trained on.

6.12 Summary of Results

Read together, Sections 6.1-6.10 establish, in order of how directly each result bears on this project's central research question (Chapter 3): the stress-test harness reproduces already-established in-distribution coverage before any out-of-range claim is examined (Section 6.1); coverage under the gradient-boosting surrogate collapses substantially and monotonically (with one explained exception) once the test distribution moves past the training boundary, traced directly to tree-based models' inability to extrapolate past their training range (Sections 6.2-6.3); an architecturally different surrogate (the MLP), matched on in-distribution accuracy, collapses faster and more severely under the identical stress test despite - or, as this chapter's central finding establishes, because of - its ability to keep extrapolating (Sections 6.4-6.5); this inversion of the naive intuition is traced to a specific, verified mechanism, a true relationship that saturates due to right-censoring at the simulated day boundary, against which a frozen prediction happens to be a better approximation than an unconstrained linear extrapolation (Section 6.6); one target's apparent partial coverage recovery at extreme severity is shown to be a data-generating-process artifact of the same censoring mechanism rather than genuine interval reliability (Section 6.7); Mondrian conformal prediction's per-category structure, shown elsewhere in this project to correct a real within-distribution conditional coverage gap, does not meaningfully protect against this

out-of-distribution failure, since its categories and quantiles are equally derived from, and therefore equally blind beyond, the in-range calibration data (Section 6.8); the failure mode is at least measurably detectable rather than silent, a genuine if partial practical mitigation (Section 6.9); and the finding survives direct consideration of the most plausible alternative explanations (Section 6.10).

CHAPTER 7

Conclusion and Future Scope

7.1 Summary of Findings

This report set out to test, in a discrete-event queueing simulation domain, the second of two limitations that Gopakumar et al. (2026) explicitly flag as untested in their own validation of conformal prediction for surrogate-model uncertainty quantification: that conformal prediction's coverage guarantee depends on an exchangeability assumption between calibration and test data, and is not expected to survive a violation of that assumption. The answer, established across Chapter 6, is that this limitation is confirmed, and additionally depends on surrogate architecture in a specific, counter-intuitive way not previously documented in this project's literature review: coverage collapses under both a gradient-boosting surrogate and a structurally different multilayer perceptron surrogate once the test distribution's demand level moves past the training boundary, but the architecture capable of extrapolating (the MLP) fails faster and more severely than the one that cannot (gradient boosting), because the true relationship in this domain saturates under extreme demand - a consequence of a disclosed, justified censoring mechanism in how the simulation counts completed visits - and gradient boosting's frozen, non-extrapolating prediction happens to approximate a genuinely flat true function better than the MLP's confident, unconstrained continuation of an upward trend that does not, in fact, continue.

7.2 Contribution and Novelty

This project's contribution, assessed honestly against the research gap stated in Chapter 3, is twofold. First, it provides a concrete, domain-specific confirmation of Gopakumar et al.'s exchangeability limitation outside the physics-simulation domains in which it was originally raised as an untested concern - to the extent this project's 30-paper literature review (Chapter 2) was able to establish, the first such test in a discrete-event queueing domain. Second, and more novel, it identifies and verifies a specific mechanism by which a surrogate architecture's extrapolation capability can be actively counterproductive under distribution

shift, rather than simply insufficiently protective: an architecture that cannot extrapolate defaults, in effect, to predicting "no further change," which is a better approximation of a saturating true relationship than an architecture that confidently continues a learned trend the true relationship does not sustain. This result is presented not as a general claim that tree ensembles are safer than neural networks under distribution shift - Section 6.6.1 states explicitly that the opposite would very plausibly hold in a domain where the true relationship continues growing rather than saturating - but as evidence that extrapolation capability's value under distribution shift is conditional on the true relationship's own behavior past the training range, a consideration this project's literature review did not find already established as a practical guideline in the conformal prediction or surrogate-modeling literature it surveyed.

7.3 Limitations

Several limitations of this work are stated here explicitly rather than left implicit. First, as in the companion report, the discrete-event simulation's service-time distributions are calibrated from literature-standard parameters by triage-acuity level rather than derived from the real dataset used in this project, since that dataset contains no length-of-stay field; the stress test's findings are therefore findings about a simulation only partially validated against real hospital data. Second, only two surrogate architectures were tested; while they are structurally quite different (a tree ensemble and a feedforward neural network), other architecture families - Gaussian processes with informative priors, recurrent or attention-based sequence models operating on the DES's underlying event stream rather than scalar scenario summaries - were not tested and might extrapolate differently again. Third, only one direction and one type of distribution shift was tested (arrival-rate multiplier increasing past its training range, with staffing capacity held to its normal distribution); a shift in staffing capacity, in the joint relationship between the two covariates, or in a covariate the DES does not model at all (a genuinely novel patient-mix or triage-acuity shift) could plausibly produce different failure characteristics. Fourth, the specific MLP architecture and hyperparameters used were not systematically varied (Section 6.10); a differently regularized or configured network might extrapolate less aggressively, though the qualitative mechanism identified (continuing a learned trend absent a true relationship that sustains it) is expected to be a

general property of feedforward neural networks rather than specific to this project's particular configuration.

7.4 Future Scope

Several directions follow naturally from this project's findings and limitations. Testing additional surrogate architectures - particularly ones with explicit mechanisms for recognizing or flagging out-of-distribution inputs, such as Gaussian processes whose predictive variance naturally grows away from training data, or deep ensembles (Section 2.5) whose member disagreement can itself serve as a shift indicator - would extend this chapter's two-architecture comparison into a broader map of which architectural properties help, hurt, or are neutral under this specific kind of distribution shift. Conformal prediction methods explicitly designed for covariate shift and non-exchangeable data (Section 2.6, Tibshirani et al., 2019; Barber et al., 2023) were outside this report's scope but represent a direct, literature-grounded next step for recovering some coverage guarantee under exactly the kind of shift studied here, rather than only documenting its failure. Testing a shift in staffing capacity, or a joint shift in both covariates simultaneously, would determine whether the specific saturation-driven mechanism identified in Section 6.6 is particular to arrival-rate shift or a more general property of this simulation's near-capacity behavior. Finally, combining this report's exchangeability finding with the companion report's Mondrian conditional-coverage finding - testing whether a richer Mondrian taxonomy, or a conformal method explicitly combining category-conditional and shift-aware calibration, offers any partial protection under a shift milder than the severe one studied here - is a natural next question raised by, but not answered within, either of this project's two reports.

CHAPTER 8

References

[1] Vovk, V., Gammerman, A., Shafer, G. (2005). Algorithmic Learning in a Random World. Springer.

[2] Papadopoulos, H., Proedrou, K., Vovk, V., Gammerman, A. (2002). Inductive Confidence Machines for Regression. ECML 2002, LNCS 2430, pp. 345-356.

[3] Shafer, G., Vovk, V. (2008). A Tutorial on Conformal Prediction. Journal of Machine Learning Research, 9, 371-421.

[4] Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R.J., Wasserman, L. (2018). Distribution-Free Predictive Inference for Regression. JASA, 113(523), 1094-1111.

[5] Tibshirani, R.J., Barber, R.F., Candès, E., Ramdas, A. (2019). Conformal Prediction Under Covariate Shift. NeurIPS 32.

[6] Barber, R.F., Candès, E., Ramdas, A., Tibshirani, R.J. (2023). Conformal Prediction Beyond Exchangeability. Annals of Statistics, 51(2), 816-845.

[7] Angelopoulos, A., Bates, S. (2021). A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511.

[8] Romano, Y., Patterson, E., Candès, E. (2019). Conformalized Quantile Regression. NeurIPS 32.

[9] Vovk, V., Lindsay, D., Nouretdinov, I., Gammerman, A. (2003). Mondrian Confidence Machine. Technical Report, Royal Holloway, University of London.

[10] Boström, H., Johansson, U. (2020). Mondrian Conformal Regressors. PMLR 128 (COPA 2020), pp. 114-133.

[11] Gopakumar, V. et al. (2026). Uncertainty Quantification of Surrogate Models Using Conformal Prediction. Machine Learning: Science and Technology.

[12] Friedman, J.H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), 1189-1232.

[13] Hastie, T., Tibshirani, R., Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.), Chapter 9-10 (Trees, Boosting). Springer.

[14] Rasmussen, C.E., Williams, C.K.I. (2006). Gaussian Processes for Machine Learning. MIT Press.

[15] Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep Learning, Chapter 6 (Deep Feedforward Networks). MIT Press.

[16] Lakshminarayanan, B., Pritzel, A., Blundell, C. (2017). Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. NeurIPS 30, 6402-6413.

[17] Abdar, M. et al. (2021). A Review of Uncertainty Quantification in Deep Learning: Techniques, Applications and Challenges. Information Fusion, 76, 243-297.

[18] Kennedy, M.C., O'Hagan, A. (2001). Bayesian Calibration of Computer Models. JRSS-B, 63(3), 425-464.

[19] Xu, K. et al. (2021). How Neural Networks Extrapolate: From Feedforward to Graph Neural Networks. ICLR 2021.

[20] Green, L.V., Soares, J., Giglio, J.F., Green, R.A. (2006). Using Queueing Theory to Increase the Effectiveness of Emergency Department Provider Staffing. Academic Emergency Medicine, 13(1), 61-68.

[21] Hu, X. et al. (2018). Applying Queueing Theory to the Study of Emergency Department Operations: A Survey and a Discussion of Comparable Simulation Studies. International Transactions in Operational Research.

[22] A Simulation-Based Optimization Approach for the Calibration of a Discrete Event Simulation Model of an Emergency Department. arXiv:2102.00945 (2021).

[23] Discrete Event Simulation for Emergency Department Modelling: A Systematic Review of Validation Methods. ScienceDirect (2022).

[24] A Simulation-Based Optimization Approach for Analyzing the Ambulance Diversion Phenomenon in an Emergency Department Network. arXiv:2108.04162.

[25] Machine Learning-Based Prediction of Hospital Prolonged Length of Stay Admission at Emergency Department: A Gradient Boosting Algorithm Analysis. Frontiers in Artificial Intelligence (2023).

[26] An Artificial Intelligence-Based Framework for Predicting Emergency Department Overcrowding: Development and Evaluation Study. arXiv:2504.18578 (2025).

[27] Hospital Triage and Patient History Data. Kaggle (user maalona) - Yale New Haven Health System, retrospective study, March 2014 - July 2017.

APPENDIX A

Source Code Listings

The following pages list the real source code underlying this report's results, directly from this project's repository under src/. Each file is reproduced in full, unmodified, with original line numbers.

src/utils/extract_distributions.py - Real-data distribution extraction (arrivals, ESI mix)

```
"""
Week 3: extract calibration distributions for the ER DES.

Arrivals and patient volume come from the real Kaggle dataset
(data/processed/triage_data.parquet). Service/treatment time does not exist
in that dataset (no discharge timestamp, only 4-hour arrival bins), so those
come from published ED simulation literature, stratified by ESI acuity
level (1 = most urgent, 5 = least urgent).

Service time source: log-normal parameters commonly used in ED DES studies,
e.g. Ahalt et al. (2018) and similar ED queueing simulation papers that
report mean/SD treatment time by ESI level.

The dataset combines three EDs (`dep_name` A/B/C: one academic, two
community — per the Kaggle dataset description) collected March 2014 -
July 2017 (1248 days). Our DES models one ED, so we calibrate on a single
department (default: A, the largest/academic site) rather than the pooled
total — pooling all three sites would overstate one ED's real arrival rate.
"""

import pandas as pd

RAW = "data/processed/triage_data.parquet"
OUT_TABLES = "results/tables"
OUT_FIGURES = "results/figures"

DEPARTMENT = "A"
STUDY_PERIOD_DAYS = 1248  # March 2014 - July 2017, per Kaggle dataset description

# (mean_minutes, sd_minutes) per ESI level, log-normal, from ED sim literature
SERVICE_TIME_PARAMS_BY_ESI = {
    1: (180, 90),
    2: (150, 75),
    3: (120, 60),
    4: (75, 40),
    5: (45, 25),
}


def load_data(department=DEPARTMENT):
    df = pd.read_parquet(RAW)
    return df[df["dep_name"] == department]


def arrival_distribution(df):
    """Real arrival counts by 4-hour bin, day of week, and month, for one ED."""
    by_hour_bin = df["arrivalhour_bin"].value_counts().sort_index()
```

```
    by_day = df["arrivalday"].value_counts()
    by_month = df["arrivalmonth"].value_counts()
    return by_hour_bin, by_day, by_month


def esi_distribution(df):
    """Real ESI (acuity) mix — drives which service-time distribution applies."""
    return df["esi"].dropna().astype(int).value_counts(normalize=True).sort_index()


def main():
    df = load_data()
    visits_per_day = len(df) / STUDY_PERIOD_DAYS

    by_hour_bin, by_day, by_month = arrival_distribution(df)
    esi_mix = esi_distribution(df)

    by_hour_bin.to_csv(f"{OUT_TABLES}/arrivals_by_hour_bin.csv", header=["count"])
    by_day.to_csv(f"{OUT_TABLES}/arrivals_by_day.csv", header=["count"])
    by_month.to_csv(f"{OUT_TABLES}/arrivals_by_month.csv", header=["count"])
    esi_mix.to_csv(f"{OUT_TABLES}/esi_mix.csv", header=["proportion"])

    service_time_df = pd.DataFrame(
        SERVICE_TIME_PARAMS_BY_ESI, index=["mean_minutes", "sd_minutes"]
    ).T
    service_time_df.index.name = "esi"
    service_time_df.to_csv(f"{OUT_TABLES}/service_time_params.csv")

    pd.Series(
        {"department": DEPARTMENT, "study_period_days": STUDY_PERIOD_DAYS,
         "total_visits": len(df), "visits_per_day": visits_per_day},
    ).to_csv(f"{OUT_TABLES}/calibration_meta.csv", header=["value"])

    print(f"Department {DEPARTMENT}: {len(df)} visits over {STUDY_PERIOD_DAYS} days"
          f" = {visits_per_day:.1f} visits/day\n")
    print("Arrivals by 4h bin:\n", by_hour_bin, "\n")
    print("Arrivals by day:\n", by_day, "\n")
    print("Arrivals by month:\n", by_month, "\n")
    print("ESI mix:\n", esi_mix, "\n")
    print("Service time params (literature, by ESI):\n", service_time_df)


if __name__ == "__main__":
    main()
```

src/des/er_simulation.py - SimPy discrete-event ER simulation

```
"""
Week 4-5: ER discrete-event simulation, calibrated on real arrival/ESI
distributions (results/tables/, department A, see extract_distributions.py)
and literature service-time params.

Single resource pool (`capacity`) represents a combined bed+provider slot for
the full visit duration — the standard simplification in ED queueing/DES
literature (M/G/c queue), used here because we have no real data to split
staffing into separate doctor/bed/nurse ratios; modeling them separately
would just add unverified guesses without adding insight for this project's
actual question (UQ methodology, not hospital operations realism).

`n_capacity` is a parameter, not fixed — Week 6-7 varies it (along with
arrival-rate scaling) to generate surrogate training data across scenarios.

Default n_capacity=30: offered load = arrival_rate * mean_service_time
```

```
(Erlangs) is ~22 erlangs on average and ~34 erlangs during the 11-14 peak
bin (department A: ~258 visits/day, ~121 min mean service time). 30 servers
sits between those, giving a stable but visibly congested system — useful
for a DES whose whole purpose is producing realistic queueing/wait
variation, not a system with near-zero wait.
"""

import numpy as np
import pandas as pd
import simpy

TABLES = "results/tables"

# Literature log-normal service time params by ESI, from extract_distributions.py
SERVICE_TIME_PARAMS_BY_ESI = {
    1: (180, 90),
    2: (150, 75),
    3: (120, 60),
    4: (75, 40),
    5: (45, 25),
}

STUDY_PERIOD_DAYS = 1248  # March 2014 - July 2017, per extract_distributions.py

HOUR_BINS = ["23-02", "03-06", "07-10", "11-14", "15-18", "19-22"]

# "23-02" wraps midnight (hours 23,0,1,2) — build the hour->bin map explicitly
# rather than hour // 4, which would wrongly put hours 0-3 in "23-02".
HOUR_TO_BIN = {}
for _bin_label, _hours in zip(
    HOUR_BINS, [[23, 0, 1, 2], [3, 4, 5, 6], [7, 8, 9, 10], [11, 12, 13, 14], [15, 16, 17, 18],
[19, 20, 21, 22]]
):
    for _h in _hours:
        HOUR_TO_BIN[_h] = _bin_label


def lognormal_params_from_mean_sd(mean, sd):
    """Convert a desired (mean, sd) in minutes to the underlying normal's (mu, sigma)."""
    variance = sd**2
    mu = np.log(mean**2 / np.sqrt(variance + mean**2))
    sigma = np.sqrt(np.log(1 + variance / mean**2))
    return mu, sigma


def load_calibration(tables_dir=TABLES):
    """Real arrival-by-hour-bin distribution and real ESI mix (single department).

    tables_dir defaults to the department-A tables used throughout the project;
    pass a different directory (e.g. results/tables/dept_b) to calibrate the
    same DES logic against a different department's real distributions without
    touching department A's files.
    """
    arrivals = pd.read_csv(f"{tables_dir}/arrivals_by_hour_bin.csv", index_col=0)["count"]
    arrivals = arrivals.reindex(HOUR_BINS)
    esi_mix = pd.read_csv(f"{tables_dir}/esi_mix.csv", index_col=0)["proportion"]
    return arrivals, esi_mix


class ERSimulation:
    """
    One 24h day of ER operation. Arrival rate varies by 4-hour bin (real data),
    ESI is sampled from the real acuity mix, service time is sampled per-ESI
    log-normal (literature params). Higher-acuity (lower ESI number) patients
    get priority for the shared capacity pool, matching real triage behavior.
    """

    def __init__(self, n_capacity, arrivals_by_bin, esi_mix, arrival_rate_multiplier=1.0,
seed=None):
```

```
  84          self.n_capacity = n_capacity
  85          self.arrivals_by_bin = arrivals_by_bin
  86          self.esi_mix = esi_mix
  87          self.np_rng = np.random.default_rng(seed)
  88
  89          total_visits_per_day = arrivals_by_bin.sum() / STUDY_PERIOD_DAYS
  90          self.rate_per_bin = (
  91              (arrivals_by_bin / arrivals_by_bin.sum()) * total_visits_per_day / 4 *
arrival_rate_multiplier
  92          )  # per hour
  93
  94          self.wait_times = []
  95          self.total_times = []
  96          self.n_patients = 0
  97
  98      def patient(self, env, capacity):
  99          arrival_time = env.now
 100          esi = self.np_rng.choice(self.esi_mix.index.astype(int), p=self.esi_mix.values)
 101          mean, sd = SERVICE_TIME_PARAMS_BY_ESI[esi]
 102          mu, sigma = lognormal_params_from_mean_sd(mean, sd)
 103          service_time = self.np_rng.lognormal(mu, sigma)
 104
 105          with capacity.request(priority=esi) as req:
 106              yield req
 107              wait = env.now - arrival_time
 108              self.wait_times.append(wait)
 109              yield env.timeout(service_time)
 110              self.total_times.append(env.now - arrival_time)
 111              self.n_patients += 1
 112
 113      def arrival_process(self, env, capacity):
 114          for hour in range(24):
 115              bin_idx = HOUR_TO_BIN[hour]
 116              rate = self.rate_per_bin[bin_idx]
 117              for _ in range(60):  # simulate minute-by-minute within this hour
 118                  if self.np_rng.random() < rate / 60:
 119                      env.process(self.patient(env, capacity))
 120                  yield env.timeout(1)
 121
 122      def run(self):
 123          env = simpy.Environment()
 124          capacity = simpy.PriorityResource(env, capacity=self.n_capacity)
 125          env.process(self.arrival_process(env, capacity))
 126          env.run(until=24 * 60)
 127          return {
 128              "n_patients": self.n_patients,
 129              "mean_wait_minutes": np.mean(self.wait_times) if self.wait_times else 0.0,
 130              "mean_total_minutes": np.mean(self.total_times) if self.total_times else 0.0,
 131              "p95_wait_minutes": np.percentile(self.wait_times, 95) if self.wait_times else 0.0,
 132          }
 133
 134
 135  def run_scenario(n_capacity=30, arrival_rate_multiplier=1.0, seed=None, tables_dir=TABLES):
 136      arrivals_by_bin, esi_mix = load_calibration(tables_dir)
 137      sim = ERSimulation(n_capacity, arrivals_by_bin, esi_mix, arrival_rate_multiplier, seed=seed)
 138      return sim.run()
 139
 140
 141  if __name__ == "__main__":
 142      result = run_scenario(seed=42)
 143      print(result)
```

src/des/validate.py - DES validation against real daily volume

```python
"""
Week 4-5: validate the calibrated DES against real aggregated stats.

Runs many independent simulated days and compares mean simulated patients/day
to the real department-A daily average (calibration_meta.csv). Some
undershoot is expected: patients still queued when a 24h simulated day ends
are cut off (right-censoring at the day boundary) rather than carried over,
since each run models one independent day for later surrogate-training
sampling (Week 6-7), not a continuous multi-day rollout.
"""

import pandas as pd

from src.des.er_simulation import run_scenario

N_DAYS = 200
N_CAPACITY = 30


def main():
    results = [run_scenario(n_capacity=N_CAPACITY, seed=s) for s in range(N_DAYS)]
    df = pd.DataFrame(results)

    meta = pd.read_csv("results/tables/calibration_meta.csv", index_col=0)
    real_visits_per_day = float(meta.loc["visits_per_day", "value"])
    sim_visits_per_day = df["n_patients"].mean()

    summary = pd.Series(
        {
            "n_days_simulated": N_DAYS,
            "n_capacity": N_CAPACITY,
            "real_visits_per_day": real_visits_per_day,
            "sim_mean_visits_per_day": sim_visits_per_day,
            "sim_std_visits_per_day": df["n_patients"].std(),
            "ratio_sim_over_real": sim_visits_per_day / real_visits_per_day,
            "sim_mean_wait_minutes": df["mean_wait_minutes"].mean(),
            "sim_mean_total_minutes": df["mean_total_minutes"].mean(),
            "sim_mean_p95_wait_minutes": df["p95_wait_minutes"].mean(),
        }
    )
    summary.to_csv("results/tables/des_validation.csv", header=["value"])
    print(summary)


if __name__ == "__main__":
    main()
```

src/surrogate/generate_training_data.py - DES scenario sweep for surrogate training data

```python
"""
Week 6-7: run the calibrated DES across varied staffing/arrival scenarios to
generate surrogate training data.

Each row = one independent simulated day at a randomly sampled
(n_capacity, arrival_rate_multiplier), with its own random seed so the DES's
intrinsic stochasticity (arrival randomness, ESI mix, service-time variance)
stays in the labels. That residual variability is exactly what the later UQ
step (GP baseline / conformal prediction) needs to characterize — smoothing
it out here would defeat the point.

n_capacity range (15-45) and arrival_rate_multiplier range (0.8-1.3) span
under- to over-provisioned staffing and below/above-average demand days,
while staying inside "normal operations" — the Week 13 surge-day stress
```

```
test intentionally goes outside this range later to test exchangeability.
"""

import numpy as np
import pandas as pd

from src.des.er_simulation import run_scenario

N_SCENARIOS = 5000
N_CAPACITY_RANGE = (15, 45)
ARRIVAL_MULTIPLIER_RANGE = (0.8, 1.3)
OUT_PATH = "data/processed/surrogate_training_data.parquet"


def generate(n_scenarios=N_SCENARIOS, seed=0, seed_offset=0,
             n_capacity_range=N_CAPACITY_RANGE,
arrival_multiplier_range=ARRIVAL_MULTIPLIER_RANGE,
             tables_dir=None):
    """seed_offset shifts the per-scenario DES seeds so a second call (e.g. for
    CP calibration data) draws fully independent randomness from the first,
    on top of already using a different `seed` for the parameter sampling.

    n_capacity_range/arrival_multiplier_range/tables_dir default to department
    A's values (module constants / er_simulation.py's own default) so existing
    callers are unaffected; pass different values to calibrate against a
    different department (see src/generalization/)."""
    rng = np.random.default_rng(seed)
    n_capacity = rng.integers(n_capacity_range[0], n_capacity_range[1] + 1, size=n_scenarios)
    arrival_mult = rng.uniform(*arrival_multiplier_range, size=n_scenarios)

    run_kwargs = {} if tables_dir is None else {"tables_dir": tables_dir}
    rows = []
    for i in range(n_scenarios):
        result = run_scenario(
            n_capacity=int(n_capacity[i]),
            arrival_rate_multiplier=float(arrival_mult[i]),
            seed=seed_offset + i,
            **run_kwargs,
        )
        rows.append(
            {
                "n_capacity": n_capacity[i],
                "arrival_rate_multiplier": arrival_mult[i],
                **result,
            }
        )
    return pd.DataFrame(rows)


def main():
    df = generate()
    df.to_parquet(OUT_PATH, index=False)
    print(f"Generated {len(df)} scenarios -> {OUT_PATH}")
    print(df.describe())


if __name__ == "__main__":
    main()
```

src/surrogate/train_surrogate.py - Gradient-boosting surrogate training

```
"""
Week 6-7: train a surrogate model on the DES scenario data.
```

```
 3
 4  Gradient boosting (sklearn's HistGradientBoostingRegressor), not a neural
 5  net — this is a 2-input tabular regression problem with ~5000 rows, where
 6  gradient boosting is the standard strong baseline and needs no architecture
 7  tuning or scaling. A NN would add complexity without a corresponding benefit
 8  for this problem size/shape.
 9
10  One model per target metric (n_patients, mean_wait_minutes,
11  mean_total_minutes, p95_wait_minutes) trained independently — simpler than a
12  true multi-output model and fine here since sklearn doesn't support
13  multi-output HGBR natively.
14  """
15
16  import joblib
17  import pandas as pd
18  from sklearn.ensemble import HistGradientBoostingRegressor
19  from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
20  from sklearn.model_selection import train_test_split
21
22  DATA_PATH = "data/processed/surrogate_training_data.parquet"
23  MODELS_DIR = "models"
24  METRICS_PATH = "results/tables/surrogate_metrics.csv"
25
26  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
27  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes", "p95_wait_minutes"]
28
29
30  def main():
31      df = pd.read_parquet(DATA_PATH)
32      X_train, X_test, train_idx, test_idx = train_test_split(
33          df[FEATURES], df.index, test_size=0.2, random_state=42
34      )
35
36      metrics = []
37      for target in TARGETS:
38          y_train = df.loc[train_idx, target]
39          y_test = df.loc[test_idx, target]
40
41          model = HistGradientBoostingRegressor(random_state=42)
42          model.fit(X_train, y_train)
43          y_pred = model.predict(X_test)
44
45          metrics.append(
46              {
47                  "target": target,
48                  "mae": mean_absolute_error(y_test, y_pred),
49                  "rmse": mean_squared_error(y_test, y_pred) ** 0.5,
50                  "r2": r2_score(y_test, y_pred),
51              }
52          )
53          joblib.dump(model, f"{MODELS_DIR}/surrogate_{target}.joblib")
54
55      metrics_df = pd.DataFrame(metrics)
56      metrics_df.to_csv(METRICS_PATH, index=False)
57      print(metrics_df)
58
59
60  if __name__ == "__main__":
61      main()
```

src/surrogate/train_mlp_surrogate.py - MLP surrogate training (second architecture)

```python
"""
Publication-rigor upgrade, part 3: a second surrogate architecture.

Week 13's exchangeability finding traced the coverage collapse to a specific
mechanism: HistGradientBoostingRegressor (tree-based) cannot extrapolate
past its training range, so its prediction literally freezes for any input
outside [0.8, 1.3]. That's a real, verified mechanism - but it's specific to
tree ensembles. A neural network doesn't have the same "frozen leaf value"
limitation; its predictions keep changing outside the training range
(usually roughly linearly in the final layer, though not necessarily
*correctly*). Training an MLP surrogate here to re-run the exchangeability
stress test against and see whether the coverage collapse is a property of
CP-wrapping-an-unreliable-model in general, or specific to gradient
boosting's exact failure mode.

Same train/test split (random_state=42) as train_surrogate.py, so this is
trained on identical data to the original surrogate - a fair architecture
comparison, not a differently-set-up experiment. MLPs are scale-sensitive
(unlike tree ensembles), so each target gets a Pipeline(StandardScaler,
MLPRegressor) - the scaler is fit only on training data and saved as part of
the pipeline, so calibration/test/stress-test data downstream doesn't need
its own separate scaling step.
"""

import joblib
import pandas as pd
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

DATA_PATH = "data/processed/surrogate_training_data.parquet"
MODELS_DIR = "models"
METRICS_PATH = "results/tables/mlp_surrogate_metrics.csv"

FEATURES = ["n_capacity", "arrival_rate_multiplier"]
TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes", "p95_wait_minutes"]


def main():
    df = pd.read_parquet(DATA_PATH)
    X_train, X_test, train_idx, test_idx = train_test_split(
        df[FEATURES], df.index, test_size=0.2, random_state=42
    )

    metrics = []
    for target in TARGETS:
        y_train = df.loc[train_idx, target]
        y_test = df.loc[test_idx, target]

        model = make_pipeline(
            StandardScaler(),
            MLPRegressor(
                hidden_layer_sizes=(64, 64),
                activation="relu",
                early_stopping=True,
                n_iter_no_change=20,
                max_iter=2000,
                random_state=42,
            ),
        )
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        metrics.append(
            {
                "target": target,
                "mae": mean_absolute_error(y_test, y_pred),
```

```
                "rmse": mean_squared_error(y_test, y_pred) ** 0.5,
                "r2": r2_score(y_test, y_pred),
            }
        )
        joblib.dump(model, f"{MODELS_DIR}/surrogate_{target}_mlp.joblib")

    metrics_df = pd.DataFrame(metrics)
    metrics_df.to_csv(METRICS_PATH, index=False)
    print(metrics_df)


if __name__ == "__main__":
    main()
```

src/uq/generate_calibration_data.py - Disjoint DES scenario pool for CP calibration

```
"""
Week 9-10: generate a calibration set for conformal prediction.

CP calibration data must be separate from what the surrogate was trained on
- reusing training-set residuals would understate the true residual spread
(the surrogate fits those points), which would break coverage validity. This
uses a different parameter-sampling seed AND a large DES seed offset from
generate_training_data.py, so nothing here overlaps with the training/test
data used to fit and evaluate the surrogate and GP baseline.

Same scenario ranges as training data (n_capacity 15-45, arrival_rate_multiplier
0.8-1.3) - calibration data has to be exchangeable with the test set, which
means drawn from the same distribution, not a different one.
"""

from src.surrogate.generate_training_data import generate

N_CALIBRATION = 1200
OUT_PATH = "data/processed/cp_calibration_data.parquet"


def main():
    df = generate(n_scenarios=N_CALIBRATION, seed=1, seed_offset=100_000)
    df.to_parquet(OUT_PATH, index=False)
    print(f"Generated {len(df)} calibration scenarios -> {OUT_PATH}")
    print(df.describe())


if __name__ == "__main__":
    main()
```

src/uq/standard_cp.py - Standard (pooled) split conformal prediction

```
"""
Week 9-10: standard (split) conformal prediction on the surrogate's residuals.

Uses the surrogate models already trained in Week 6-7 (models/surrogate_*.joblib)
as point predictors - CP wraps a point predictor with a calibrated interval,
it doesn't replace it. Calibration uses the separate set from
generate_calibration_data.py, never the surrogate's own training data.

```

```
Same test set (same random_state=42 split of surrogate_training_data.parquet)
and same alpha=0.1 as gp_baseline.py, so all three methods (GP, standard CP,
Mondrian CP later) are directly comparable.

Two nonconformity measures, per the roadmap's "test multiple nonconformity
measures":
  - symmetric: |y - yhat|, giving a fixed-width interval around the point
    prediction.
  - asymmetric: separate upper/lower residual quantiles (each calibrated at
    1-alpha/2, combined via a union bound for overall >=1-alpha coverage).
    Matters here because several targets (p95_wait_minutes especially) are
    right-skewed - a symmetric interval either overcovers on the left or
    undercovers on the right.
"""

import joblib
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
MODELS_DIR = "models"
METRICS_PATH = "results/tables/standard_cp_metrics.csv"

FEATURES = ["n_capacity", "arrival_rate_multiplier"]
TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes", "p95_wait_minutes"]

ALPHA = 0.1  # must match gp_baseline.py


def conformal_quantile(scores, alpha):
    """Finite-sample-corrected (1-alpha) empirical quantile of calibration scores."""
    n = len(scores)
    level = min(np.ceil((n + 1) * (1 - alpha)) / n, 1.0)
    return np.quantile(scores, level, method="higher")


def main():
    df = pd.read_parquet(TRAIN_DATA_PATH)
    _, X_test, _, test_idx = train_test_split(df[FEATURES], df.index, test_size=0.2,
random_state=42)

    cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
    X_cal = cal_df[FEATURES]

    metrics = []
    for target in TARGETS:
        model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")

        yhat_cal = model.predict(X_cal)
        y_cal = cal_df[target].values
        residual_cal = y_cal - yhat_cal

        yhat_test = model.predict(X_test)
        y_test = df.loc[test_idx, target].values

        # symmetric
        q = conformal_quantile(np.abs(residual_cal), ALPHA)
        lower_sym = yhat_test - q
        upper_sym = yhat_test + q
        covered_sym = (y_test >= lower_sym) & (y_test <= upper_sym)

        # asymmetric
        q_hi = conformal_quantile(residual_cal, ALPHA / 2)
        q_lo = conformal_quantile(-residual_cal, ALPHA / 2)
        lower_asym = yhat_test - q_lo
        upper_asym = yhat_test + q_hi
        covered_asym = (y_test >= lower_asym) & (y_test <= upper_asym)
```

```
77
78          metrics.append(
79              {
80                  "target": target,
81                  "alpha": ALPHA,
82                  "target_coverage": 1 - ALPHA,
83                  "symmetric_coverage": covered_sym.mean(),
84                  "symmetric_mean_width": (upper_sym - lower_sym).mean(),
85                  "asymmetric_coverage": covered_asym.mean(),
86                  "asymmetric_mean_width": (upper_asym - lower_asym).mean(),
87              }
88          )
89
90      metrics_df = pd.DataFrame(metrics)
91      metrics_df.to_csv(METRICS_PATH, index=False)
92      print(metrics_df)
93
94
95  if __name__ == "__main__":
96      main()
```

src/uq/mondrian_cp.py - Mondrian conformal prediction (9-category taxonomy)

```
1   """
2   Week 11-12: Mondrian conformal prediction.
3
4   Standard CP (Week 9-10) calibrates a single pooled quantile across all
5   scenarios, which only guarantees *marginal* coverage - it says nothing about
6   whether coverage holds up within a specific staffing level or arrival regime.
7   That's exactly the first limitation Gopakumar et al. (2026) flag for CP in
8   physics domains. Mondrian CP addresses it by calibrating a separate quantile
9   per category, so each category gets its own (asymptotically valid) coverage
10  guarantee instead of relying on categories averaging out to the marginal rate.
11
12  Categories: our scenario space only has two real covariates (n_capacity,
13  arrival_rate_multiplier) - there's no "shift" field in the DES output, so we
14  don't invent one. Categories are the cross of staffing tercile x arrival-rate
15  tercile (Low/Med/High x Low/Med/High = 9 cells), using bin edges derived from
16  the calibration set's own quantiles (never the test set - that would leak
17  information into the taxonomy).
18
19  Same calibration/test data, same alpha=0.1, symmetric |y - yhat| nonconformity
20  measure as standard_cp.py's symmetric variant, so this is a clean, apples-to-
21  apples comparison of pooled vs. per-category calibration - not a different
22  setup dressed up as a comparison.
23
24  For each category we compute coverage two ways on the SAME test points:
25    - "pooled" applies standard CP's single marginal quantile, evaluated
26      per-category. This shows where the marginal guarantee breaks down.
27    - "mondrian" applies that category's own quantile. This is the actual
28      per-category coverage Mondrian CP delivers.
29  """
30
31  import joblib
32  import numpy as np
33  import pandas as pd
34  from sklearn.model_selection import train_test_split
35
36  TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
37  CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
38  MODELS_DIR = "models"
39  DETAIL_PATH = "results/tables/mondrian_cp_detail.csv"
40  SUMMARY_PATH = "results/tables/mondrian_cp_summary.csv"
```

```
 41
 42  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
 43  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes", "p95_wait_minutes"]
 44
 45  ALPHA = 0.1
 46  LEVEL_NAMES = ["Low", "Med", "High"]
 47
 48
 49  def conformal_quantile(scores, alpha):
 50      n = len(scores)
 51      level = min(np.ceil((n + 1) * (1 - alpha)) / n, 1.0)
 52      return np.quantile(scores, level, method="higher")
 53
 54
 55  def assign_categories(df, cap_bounds, arr_bounds):
 56      cap_level = np.digitize(df["n_capacity"], cap_bounds)
 57      arr_level = np.digitize(df["arrival_rate_multiplier"], arr_bounds)
 58      labels = [f"staff={LEVEL_NAMES[c]}/arrival={LEVEL_NAMES[a]}" for c, a in zip(cap_level,
arr_level)]
 59      return np.array(labels)
 60
 61
 62  def main():
 63      df = pd.read_parquet(TRAIN_DATA_PATH)
 64      _, X_test, _, test_idx = train_test_split(df[FEATURES], df.index, test_size=0.2,
random_state=42)
 65      test_df = df.loc[test_idx]
 66
 67      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
 68
 69      cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
 70      arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 / 3, 2 / 3])
 71      cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
 72      test_cat = assign_categories(test_df, cap_bounds, arr_bounds)
 73      categories = sorted(set(cal_cat))
 74
 75      detail_rows = []
 76      summary_rows = []
 77      for target in TARGETS:
 78          model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
 79
 80          yhat_cal = model.predict(cal_df[FEATURES])
 81          y_cal = cal_df[target].values
 82          abs_resid_cal = np.abs(y_cal - yhat_cal)
 83
 84          yhat_test = model.predict(X_test)
 85          y_test = test_df[target].values
 86
 87          q_pooled = conformal_quantile(abs_resid_cal, ALPHA)
 88
 89          pooled_coverages, mondrian_coverages = [], []
 90          for cat in categories:
 91              cal_mask = cal_cat == cat
 92              test_mask = test_cat == cat
 93              n_cal_cat = cal_mask.sum()
 94              n_test_cat = test_mask.sum()
 95              if n_test_cat == 0:
 96                  continue
 97
 98              q_cat = conformal_quantile(abs_resid_cal[cal_mask], ALPHA)
 99
100              y_c = y_test[test_mask]
101              yhat_c = yhat_test[test_mask]
102
103              covered_pooled = (np.abs(y_c - yhat_c) <= q_pooled).mean()
104              covered_mondrian = (np.abs(y_c - yhat_c) <= q_cat).mean()
105
106              pooled_coverages.append(covered_pooled)
107              mondrian_coverages.append(covered_mondrian)
```

```
108
109             detail_rows.append(
110                 {
111                     "target": target,
112                     "category": cat,
113                     "n_cal": n_cal_cat,
114                     "n_test": n_test_cat,
115                     "pooled_coverage": covered_pooled,
116                     "pooled_width": 2 * q_pooled,
117                     "mondrian_coverage": covered_mondrian,
118                     "mondrian_width": 2 * q_cat,
119                 }
120             )
121
122         pooled_coverages = np.array(pooled_coverages)
123         mondrian_coverages = np.array(mondrian_coverages)
124         summary_rows.append(
125             {
126                 "target": target,
127                 "target_coverage": 1 - ALPHA,
128                 "pooled_coverage_min": pooled_coverages.min(),
129                 "pooled_coverage_max": pooled_coverages.max(),
130                 "pooled_coverage_range": pooled_coverages.max() - pooled_coverages.min(),
131                 "pooled_coverage_std": pooled_coverages.std(),
132                 "mondrian_coverage_min": mondrian_coverages.min(),
133                 "mondrian_coverage_max": mondrian_coverages.max(),
134                 "mondrian_coverage_range": mondrian_coverages.max() - mondrian_coverages.min(),
135                 "mondrian_coverage_std": mondrian_coverages.std(),
136             }
137         )
138
139     pd.DataFrame(detail_rows).to_csv(DETAIL_PATH, index=False)
140     summary_df = pd.DataFrame(summary_rows)
141     summary_df.to_csv(SUMMARY_PATH, index=False)
142     print(summary_df.to_string(index=False))
143
144
145 if __name__ == "__main__":
146     main()
```

src/uq/exchangeability_stress_test.py - Exchangeability stress test, gradient-boosting surrogate

```
 1 """
 2 Week 13: exchangeability stress test.
 3
 4 Standard and Mondrian CP's coverage guarantees both rely on calibration and
 5 test data being exchangeable (drawn from the same distribution). Both
 6 Week 9-10 and Week 11-12 calibrated on n_capacity in [15,45], arrival_rate_
 7 multiplier in [0.8,1.3] and evaluated on test data from the same range - so
 8 exchangeability held by construction, and both methods hit close to nominal
 9 coverage. This script keeps calibration fixed exactly as before and pushes
10 the *test* distribution progressively further outside that range - an
11 escalating "surge day" (arrival_rate_multiplier up to 3.0x normal, well past
12 the training max of 1.3x) - to find where exchangeability actually breaks
13 down and what happens to coverage when it does.
14
15 n_capacity is still sampled from the normal [15,45] range at every severity
16 level, so the shift is isolated to arrival rate - a surge in demand, not a
17 simultaneous staffing change. That keeps this a single-variable
18 extrapolation test, which is easier to read than conflating two shifted
19 variables at once.
20
21 Reuses the exact calibration quantiles and category bounds from
```

```
  22  standard_cp.py / mondrian_cp.py - no refitting - because the whole point is
  23  to see how a *fixed* calibration behaves as the test distribution moves
  24  away from it.
  25  """
  26
  27  import joblib
  28  import numpy as np
  29  import pandas as pd
  30
  31  from src.des.er_simulation import run_scenario
  32  from src.uq.mondrian_cp import LEVEL_NAMES, assign_categories, conformal_quantile
  33
  34  TRAIN_DATA_PATH = "data/processed/surrogate_training_data.parquet"
  35  CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
  36  MODELS_DIR = "models"
  37  OUT_PATH = "results/tables/exchangeability_stress_test.csv"
  38
  39  FEATURES = ["n_capacity", "arrival_rate_multiplier"]
  40  TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes", "p95_wait_minutes"]
  41
  42  ALPHA = 0.1
  43  N_CAPACITY_RANGE = (15, 45)
  44  N_PER_SEVERITY = 300
  45  # 0.8/1.3 are in-range references (should reproduce ~90% coverage); the rest
  46  # escalate past the 1.3 training max to find where coverage breaks down.
  47  SEVERITY_LEVELS = [0.8, 1.0, 1.3, 1.5, 1.8, 2.0, 2.5, 3.0]
  48
  49
  50  def generate_ood_data(arrival_multiplier, n_scenarios, seed_offset):
  51      rng = np.random.default_rng(seed_offset)
  52      n_capacity = rng.integers(N_CAPACITY_RANGE[0], N_CAPACITY_RANGE[1] + 1, size=n_scenarios)
  53      rows = []
  54      for i in range(n_scenarios):
  55          result = run_scenario(
  56              n_capacity=int(n_capacity[i]),
  57              arrival_rate_multiplier=arrival_multiplier,
  58              seed=seed_offset + i,
  59          )
  60          rows.append({"n_capacity": n_capacity[i], "arrival_rate_multiplier": arrival_multiplier,
**result})
  61      return pd.DataFrame(rows)
  62
  63
  64  def main():
  65      cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
  66      cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
  67      arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 / 3, 2 / 3])
  68      cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)
  69
  70      rows = []
  71      for level_idx, mult in enumerate(SEVERITY_LEVELS):
  72          ood_df = generate_ood_data(mult, N_PER_SEVERITY, seed_offset=200_000 + level_idx *
10_000)
  73          ood_cat = assign_categories(ood_df, cap_bounds, arr_bounds)
  74
  75          for target in TARGETS:
  76              model = joblib.load(f"{MODELS_DIR}/surrogate_{target}.joblib")
  77
  78              yhat_cal = model.predict(cal_df[FEATURES])
  79              y_cal = cal_df[target].values
  80              abs_resid_cal = np.abs(y_cal - yhat_cal)
  81              q_pooled = conformal_quantile(abs_resid_cal, ALPHA)
  82
  83              yhat_ood = model.predict(ood_df[FEATURES])
  84              y_ood = ood_df[target].values
  85              abs_resid_ood = np.abs(y_ood - yhat_ood)
  86
  87              standard_covered = abs_resid_ood <= q_pooled
  88
```

```
            mondrian_covered = np.zeros(len(ood_df), dtype=bool)
            mondrian_width = np.zeros(len(ood_df))
            for cat in set(cal_cat):
                q_cat = conformal_quantile(abs_resid_cal[cal_cat == cat], ALPHA)
                mask = ood_cat == cat
                mondrian_covered[mask] = abs_resid_ood[mask] <= q_cat
                mondrian_width[mask] = 2 * q_cat

            rows.append(
                {
                    "target": target,
                    "arrival_rate_multiplier": mult,
                    "in_training_range": mult <= 1.3,
                    "n_scenarios": len(ood_df),
                    "standard_coverage": standard_covered.mean(),
                    "standard_width": 2 * q_pooled,
                    "mondrian_coverage": mondrian_covered.mean(),
                    "mondrian_mean_width": mondrian_width.mean(),
                }
            )

    out_df = pd.DataFrame(rows)
    out_df.to_csv(OUT_PATH, index=False)
    print(out_df.to_string(index=False))


if __name__ == "__main__":
    main()
```

src/uq/exchangeability_stress_test_mlp.py - Exchangeability stress test, MLP surrogate

```
"""
Publication-rigor upgrade, part 3 (continued): re-run the Week 13
exchangeability stress test using the MLP surrogate instead of gradient
boosting, to test whether the coverage collapse outside the training range
is specific to tree-based extrapolation (frozen leaf predictions, verified
in Week 13) or a more general property of wrapping CP around an
unreliable-outside-its-domain point predictor.

Identical setup to exchangeability_stress_test.py otherwise: same
calibration data, same severity levels, same seeds - only the model files
differ (`*_mlp.joblib`). Calibration is recomputed fresh here (not reused
from the gradient-boosting run) because the MLP's residuals on the
calibration set are different from gradient boosting's - each surrogate
needs its own conformal quantile.
"""

import joblib
import numpy as np
import pandas as pd

from src.des.er_simulation import run_scenario
from src.uq.mondrian_cp import assign_categories, conformal_quantile

CALIBRATION_DATA_PATH = "data/processed/cp_calibration_data.parquet"
MODELS_DIR = "models"
OUT_PATH = "results/tables/exchangeability_stress_test_mlp.csv"

FEATURES = ["n_capacity", "arrival_rate_multiplier"]
TARGETS = ["n_patients", "mean_wait_minutes", "mean_total_minutes", "p95_wait_minutes"]

ALPHA = 0.1
N_CAPACITY_RANGE = (15, 45)
```

```
N_PER_SEVERITY = 300
SEVERITY_LEVELS = [0.8, 1.0, 1.3, 1.5, 1.8, 2.0, 2.5, 3.0]


def generate_ood_data(arrival_multiplier, n_scenarios, seed_offset):
    rng = np.random.default_rng(seed_offset)
    n_capacity = rng.integers(N_CAPACITY_RANGE[0], N_CAPACITY_RANGE[1] + 1, size=n_scenarios)
    rows = []
    for i in range(n_scenarios):
        result = run_scenario(
            n_capacity=int(n_capacity[i]),
            arrival_rate_multiplier=arrival_multiplier,
            seed=seed_offset + i,
        )
        rows.append({"n_capacity": n_capacity[i], "arrival_rate_multiplier": arrival_multiplier,
**result})
    return pd.DataFrame(rows)


def main():
    cal_df = pd.read_parquet(CALIBRATION_DATA_PATH)
    cap_bounds = np.quantile(cal_df["n_capacity"], [1 / 3, 2 / 3])
    arr_bounds = np.quantile(cal_df["arrival_rate_multiplier"], [1 / 3, 2 / 3])
    cal_cat = assign_categories(cal_df, cap_bounds, arr_bounds)

    rows = []
    for level_idx, mult in enumerate(SEVERITY_LEVELS):
        # same seed_offset scheme as the gradient-boosting version -> identical OOD scenarios
        ood_df = generate_ood_data(mult, N_PER_SEVERITY, seed_offset=200_000 + level_idx *
10_000)
        ood_cat = assign_categories(ood_df, cap_bounds, arr_bounds)

        for target in TARGETS:
            model = joblib.load(f"{MODELS_DIR}/surrogate_{target}_mlp.joblib")

            yhat_cal = model.predict(cal_df[FEATURES])
            y_cal = cal_df[target].values
            abs_resid_cal = np.abs(y_cal - yhat_cal)
            q_pooled = conformal_quantile(abs_resid_cal, ALPHA)

            yhat_ood = model.predict(ood_df[FEATURES])
            y_ood = ood_df[target].values
            abs_resid_ood = np.abs(y_ood - yhat_ood)

            standard_covered = abs_resid_ood <= q_pooled

            mondrian_covered = np.zeros(len(ood_df), dtype=bool)
            mondrian_width = np.zeros(len(ood_df))
            for cat in set(cal_cat):
                q_cat = conformal_quantile(abs_resid_cal[cal_cat == cat], ALPHA)
                mask = ood_cat == cat
                mondrian_covered[mask] = abs_resid_ood[mask] <= q_cat
                mondrian_width[mask] = 2 * q_cat

            rows.append(
                {
                    "target": target,
                    "arrival_rate_multiplier": mult,
                    "in_training_range": mult <= 1.3,
                    "n_scenarios": len(ood_df),
                    "standard_coverage": standard_covered.mean(),
                    "standard_width": 2 * q_pooled,
                    "mondrian_coverage": mondrian_covered.mean(),
                    "mondrian_mean_width": mondrian_width.mean(),
                    "yhat_capacity30": float(model.predict(pd.DataFrame(
                        [[30, mult]], columns=FEATURES))[0]),
                }
            )

```

```
    out_df = pd.DataFrame(rows)
    out_df.to_csv(OUT_PATH, index=False)
    print(out_df.to_string(index=False))


if __name__ == "__main__":
    main()
```

APPENDIX B

Supplementary Result Tables

Full severity-sweep detail (all eight arrival-rate multipliers, standard and Mondrian coverage and width) for both surrogate architectures, for completeness.

Arrival mult.	In range	Standard cov.	Standard width	Mondrian cov.	Mondrian width
0.8	True	92.0%	46.4	89.7%	38.3
1.0	True	88.0%	46.4	93.7%	49.1
1.3	True	88.0%	46.4	89.0%	50.2
1.5	False	73.7%	46.4	77.0%	50.0
1.8	False	47.3%	46.4	48.0%	50.4
2.0	False	39.0%	46.4	40.3%	49.6
2.5	False	27.0%	46.4	29.0%	50.2
3.0	False	4.7%	46.4	4.3%	50.0

Table 6. Full severity-sweep detail: mean_total_minutes, Gradient-Boosting Surrogate.

Arrival mult.	In range	Standard cov.	Standard width	Mondrian cov.	Mondrian width
0.8	True	92.7%	47.2	93.7%	29.8
1.0	True	88.0%	47.2	92.0%	41.4
1.3	True	83.0%	47.2	89.3%	51.1
1.5	False	68.7%	47.2	72.3%	50.7
1.8	False	42.7%	47.2	44.0%	51.3
2.0	False	34.3%	47.2	35.3%	49.7
2.5	False	33.3%	47.2	34.3%	51.0

3.0	False	12.3%	47.2	10.0%	50.6

Table 7. Full severity-sweep detail: mean_wait_minutes, Gradient-Boosting Surrogate.

Arrival mult.	In range	Standard cov.	Standard width	Mondrian cov.	Mondrian width
0.8	True	93.7%	43.6	93.0%	43.7
1.0	True	90.0%	43.6	90.7%	43.2
1.3	True	93.3%	43.6	92.0%	42.0
1.5	False	78.7%	43.6	77.7%	42.3
1.8	False	70.3%	43.6	67.3%	42.1
2.0	False	64.7%	43.6	62.0%	43.3
2.5	False	49.7%	43.6	45.7%	42.0
3.0	False	31.7%	43.6	31.3%	42.4

Table 8. Full severity-sweep detail: n_patients, Gradient-Boosting Surrogate.

Arrival mult.	In range	Standard cov.	Standard width	Mondrian cov.	Mondrian width
0.8	True	93.7%	362.9	95.0%	218.1
1.0	True	88.3%	362.9	92.3%	304.0
1.3	True	82.0%	362.9	92.7%	443.6
1.5	False	69.7%	362.9	79.3%	438.7
1.8	False	43.0%	362.9	56.7%	449.1
2.0	False	33.0%	362.9	40.7%	427.4
2.5	False	63.7%	362.9	71.0%	442.4
3.0	False	78.3%	362.9	86.3%	438.3

Table 9. Full severity-sweep detail: p95_wait_minutes, Gradient-Boosting Surrogate.

Arrival mult.	In range	Standard cov.	Standard width	Mondrian cov.	Mondrian width
0.8	True	92.0%	44.9	90.7%	35.9
1.0	True	88.3%	44.9	93.3%	46.5
1.3	True	88.3%	44.9	89.3%	47.8
1.5	False	42.7%	44.9	47.0%	47.7
1.8	False	2.3%	44.9	3.3%	48.0
2.0	False	0.0%	44.9	0.3%	47.5
2.5	False	0.0%	44.9	0.0%	47.8
3.0	False	0.0%	44.9	0.0%	47.7

Table 10. Full severity-sweep detail: mean_total_minutes, MLP Surrogate.

Arrival mult.	In range	Standard cov.	Standard width	Mondrian cov.	Mondrian width
0.8	True	94.0%	47.7	88.3%	29.4
1.0	True	88.3%	47.7	93.7%	42.2
1.3	True	86.7%	47.7	90.7%	52.8
1.5	False	78.3%	47.7	83.0%	52.3
1.8	False	55.0%	47.7	57.3%	53.1
2.0	False	43.3%	47.7	44.0%	51.2
2.5	False	15.7%	47.7	23.0%	52.7
3.0	False	11.0%	47.7	17.3%	52.3

Table 11. Full severity-sweep detail: mean_wait_minutes, MLP Surrogate.

Arrival mult.	In range	Standard cov.	Standard width	Mondrian cov.	Mondrian width
0.8	True	91.7%	43.2	91.0%	44.6
1.0	True	92.3%	43.2	93.7%	43.5
1.3	True	93.0%	43.2	94.7%	43.0
1.5	False	72.0%	43.2	71.7%	43.2
1.8	False	6.7%	43.2	9.0%	43.0
2.0	False	0.0%	43.2	0.0%	44.0
2.5	False	0.0%	43.2	0.0%	43.0
3.0	False	0.0%	43.2	0.0%	43.3

Table 12. Full severity-sweep detail: n_patients, MLP Surrogate.

Arrival mult.	In range	Standard cov.	Standard width	Mondrian cov.	Mondrian width
0.8	True	95.0%	353.4	95.0%	196.8
1.0	True	87.0%	353.4	91.0%	293.7
1.3	True	83.3%	353.4	90.7%	411.0
1.5	False	79.3%	353.4	84.7%	407.6
1.8	False	65.0%	353.4	67.3%	416.4
2.0	False	58.3%	353.4	61.7%	400.6
2.5	False	30.0%	353.4	34.3%	410.1
3.0	False	10.0%	353.4	18.3%	407.5

Table 13. Full severity-sweep detail: p95_wait_minutes, MLP Surrogate.